

Lab #7: Final Project

Name: Ahan Dhingra

Partners at Table: Kuval

```
In [2]: import matplotlib.pyplot as plt
import matplotlib.image as mimg
import pandas as pd
def InlineImage(img, width=8, height=8, caption=None, fs= 16):
    # Author: BG
    # Renders an image with a caption underneath using matplotlib
    # Requires matplotlib.pyplot to be imported as plt and matplotlib.image to be imp
    # img - string, the location of an image file
    # width, height - numbers, the dimensions of the figure the image is displayed
    # caption - string, the caption to be displayed beneath the image
    # fs - number, the font size of the caption
    fig,ax = plt.subplots(figsize=(width,height))
    ax.imshow(mimg.imread(img))
    ax.set_xlabel(caption,fontsize=fs)
    ax.tick_params(
        which='both',
        bottom=False,
        top=False,
        left=False,
        labelleft=False,
        labelbottom=False)
    ax.spines["top"].set_visible(False)
    ax.spines["left"].set_visible(False)
    ax.spines["right"].set_visible(False)
    ax.spines["bottom"].set_visible(False)
```

Uncertainty formula for fluctuations:

$$\frac{\max - \min}{\sqrt{3}}$$

The end goal of this lab is to build the servo motor circuit shown below.

```
In [5]: InlineImage("servocircuit.PNG", 16,16, caption = "Figure 1: Servo Motor Full Circui
```

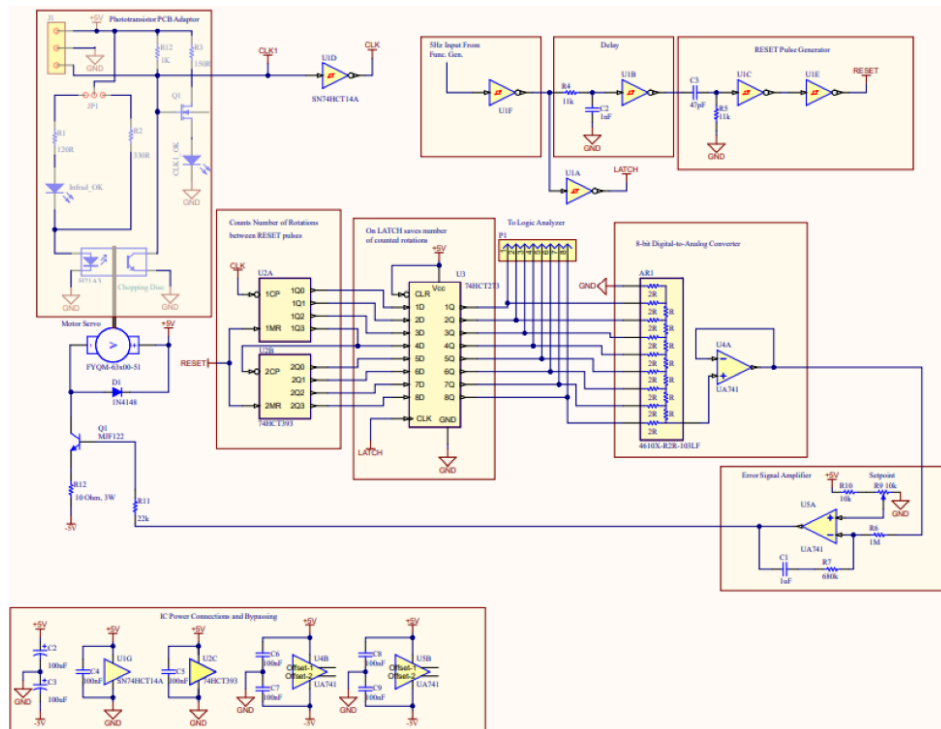


Figure 1: Servo Motor Full Circuit Diagram

This circuit is built through 5 components, all of which are built and tested individually, and will form the servo motor when connected correctly.

LATCH AND RESET Generator

To start, I will build the LATCH and RESET generator circuit, which has 2 outputs, the LATCH and RESET, shown in figure 2.

The LATCH signal is sent to the D-LATCH, telling it to store data, and the RESET is sent to the counter, which resets the counter. This circuit's job is to make sure there is a suitable delay between the when the data is latched, and when the counter is reset, since if the counter is reset before we can latch, we lose that data.

To latch the output of the counter (before it is RESET), we need to provide a square wave to the CLK input of the D-latch, which will tell it to latch its current input on the rising edge of the square wave.

I will create a 5 Hz Square wave input with my AD3, oscillating between logic level HI and LO (5 V and 0 V) respectively, invert the signal twice (U1F and U1A) to produce the LATCH output.

In [12]: `InlineImage("inverterCircuit.jpg", caption = "Figure 2: Latch and Reset Generator C`

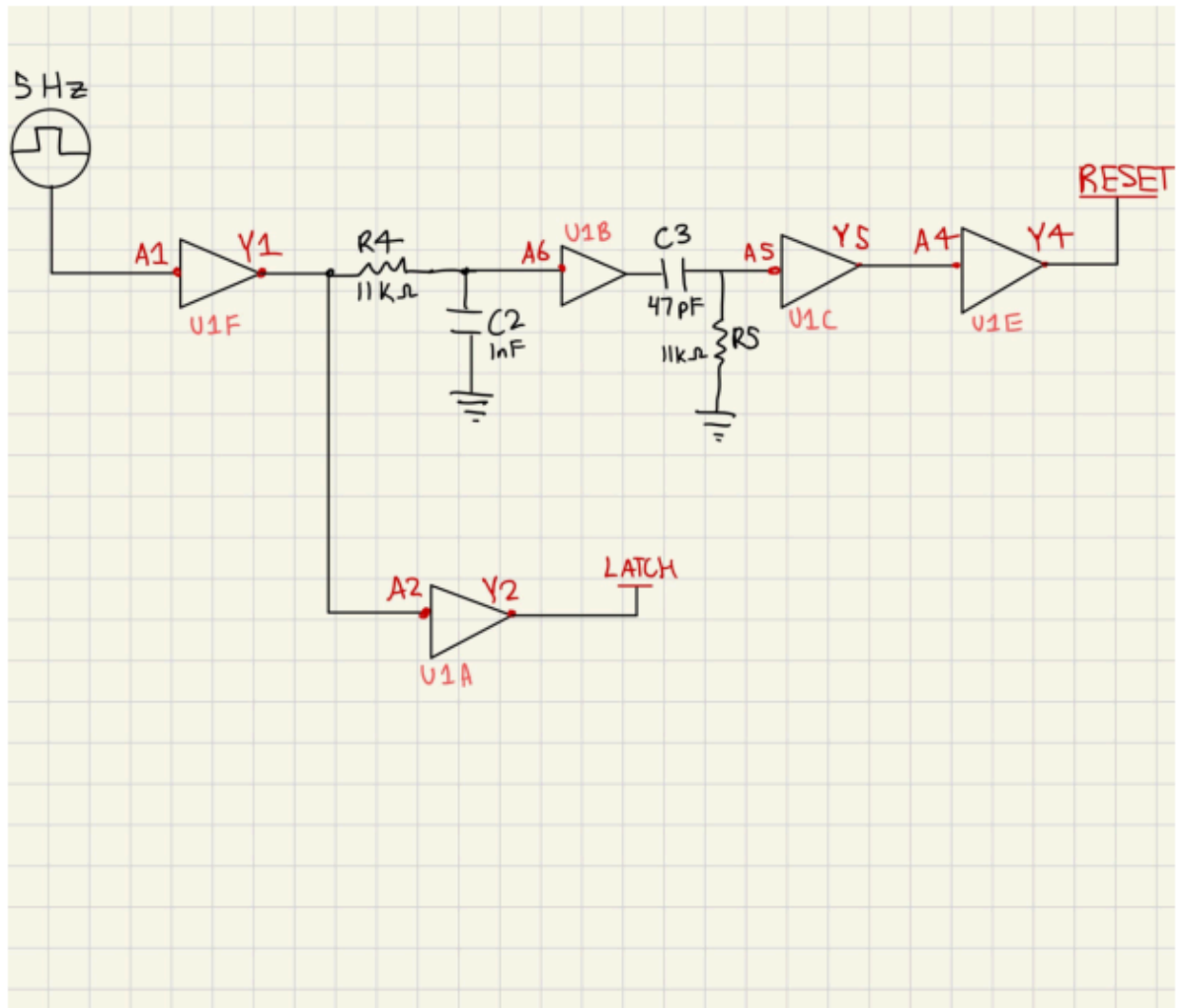
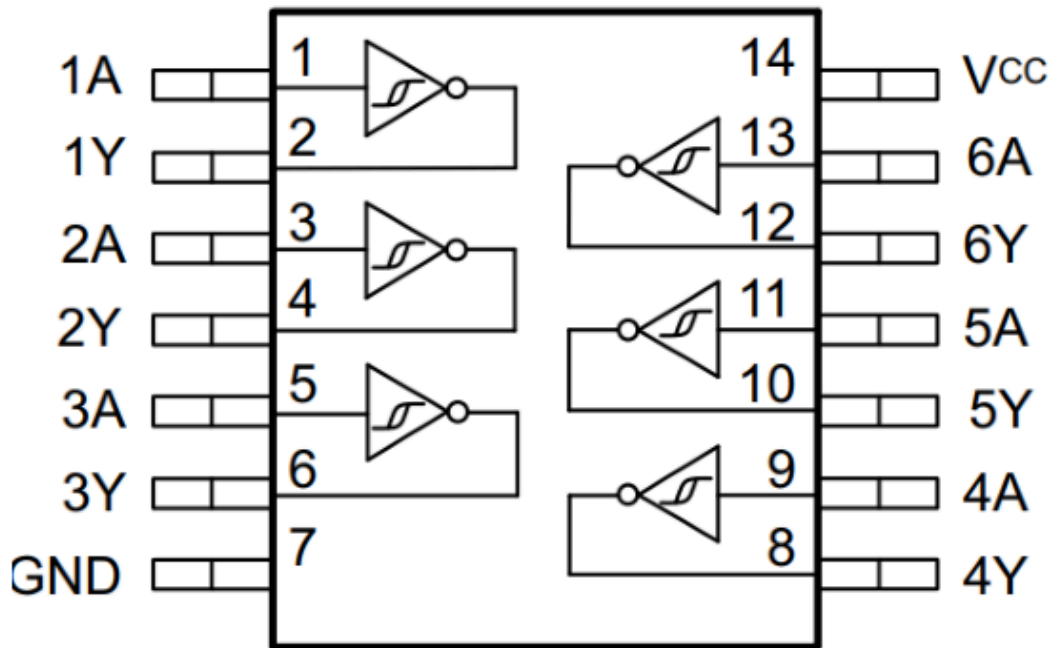


Figure 2: Latch and Reset Generator Circuit Diagram

First, I need to measure the hysteresis of the inverter I am using, which is the SN74HC14N Schmitt Trigger Inverter, with a functional pinout diagram and functional table as seen here.

In [6]: `InlineImage("inverterFD.PNG", caption = "Figure 3: SN74HC14N Logic Pinout Diagram ")`



Functional pinout

Figure 3: SN74HC14N Logic Pinout Diagram

In [16]: `InlineImage("inverterFT.PNG", caption = "Figure 4: SN74HC14N Function Table ")`

INPUT	OUTPUT
A	Y
L	H
H	L

Figure 4: SN74HC14N Function Table

The voltages at which is switches from high to low, and low to high are different voltages for a Schmitt Trigger Inverter. To measure hysteresis, I set up a triangle wave using the wave generator on my AD3, with Amplitude 2.5V, offset 2.5V (so the wave goes from 0 to 5V), matching the input voltage required by the Inverter. I used a frequency of 200 Hz, and 50% symmetry.

I set up my inverter, grounding all of the inputs except for the one I used, so that they did not cause any interference. I put the triangle wave at A1 (acting as my input), and measured

the voltage with my scope across Y1 and GND, with V+ at Y1, and V- at GND, and saw an output as such.

```
In [7]: InlineImage("HysteresisTriangleInput.PNG", 10, 10, caption = "Figure 5: Hysteresis
```

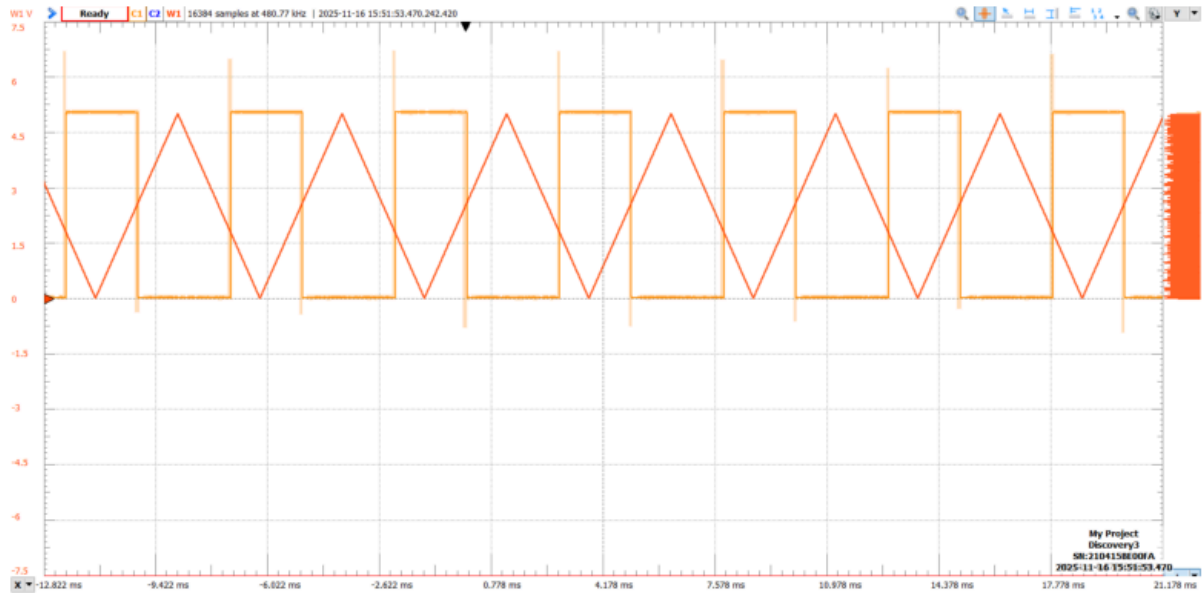


Figure 5: Hysteresis Measurement

Using the zoom and cursor tools, I measured the voltages at the transtion points (where the square wave changes value) and found my values:

For the 1->0 transition, I measured $2.57V \pm 0.02$ (overestimating the uncertainty). For 0 -> 1, I measured $1.78V \pm 0.02$

Next, I built the Latch and Reset generator Circuit shown in figure 2.

```
In [8]: InlineImage("latchRESETphoto.jpg", 11,11, caption = "Figure 6: Latch and Reset Gene
```

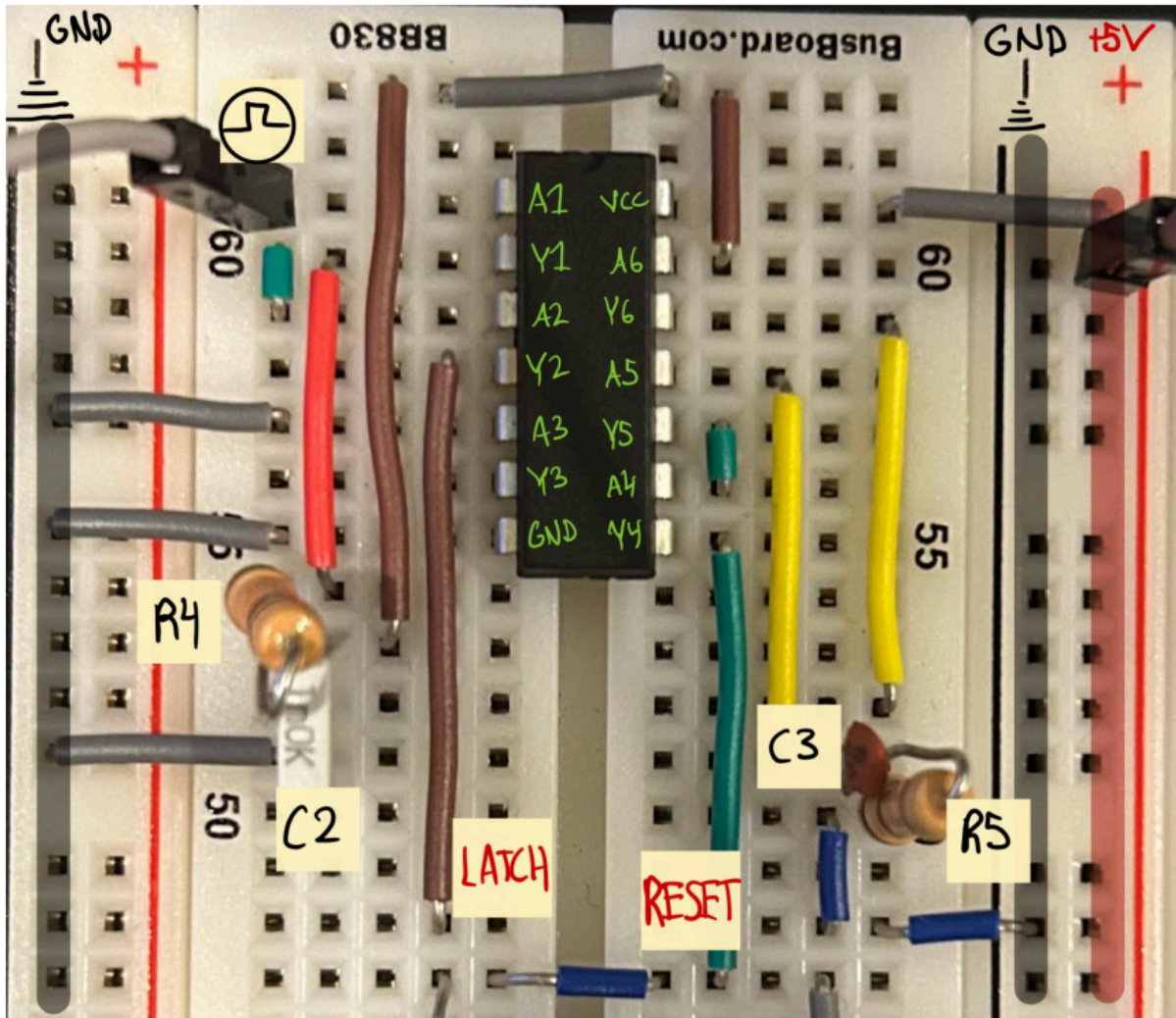


Figure 6: Latch and Reset Generator Circuit

I made sure to ground the one input I did not use, so as not to cause any cross talk between the inputs of the inverters, and I connected 5V to VCC.

Note: Every component in this circuit uses 5V or -5V for its voltage source.

I tested the delay of the circuit (this is to make sure there is a delay between when LATCH output, and the RESET output), by measuring the output after the delay (at Y6 in figure 2) and comparing it to the 5 hz square wave input at A1. I connected 1+ to Y6, and 1- to ground, and also added the square wave input to my scope, and used this to measure the delay.

In [10]: `InlineImage("delayMeasurement.png", 11,11, caption = "Figure 7: Delay Measurement "`

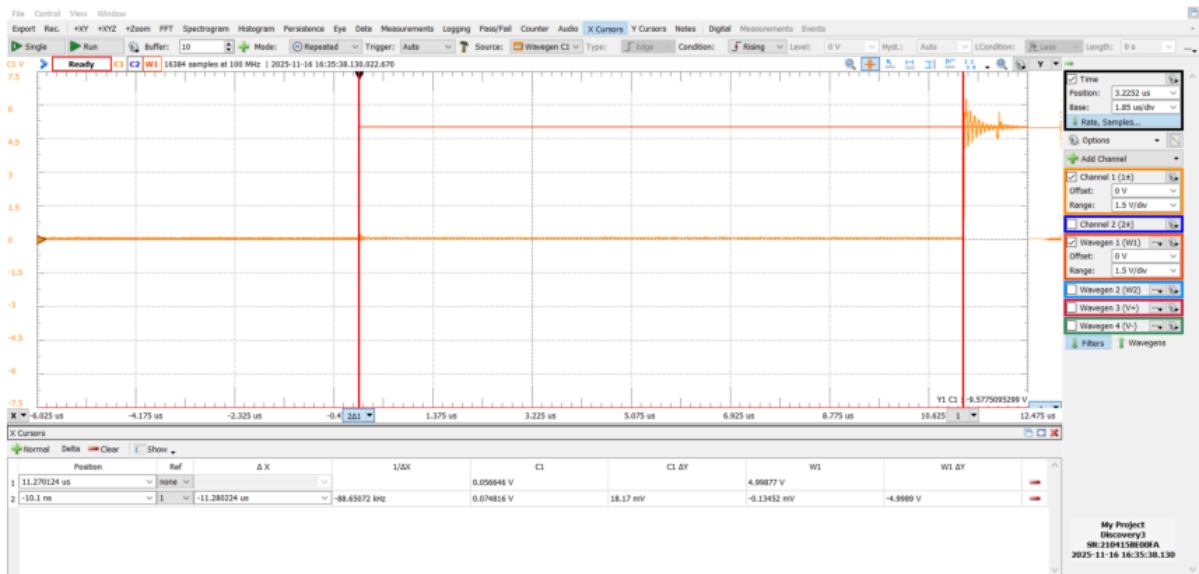


Figure 7: Delay Measurement

In the figure above, we can see that there is a delay between when the square wave input rises (on the left), and when the output at the delay rises (on the right).

Using the delta cursors, I measured a delay of $11.28\mu\text{s} \pm 0.3$, with the uncertainty accounting for fluctuations using formula 1.

The way this delay is introduced is through the RC network seen in figure 2, in between U1F and U1B. Voltage across a capacitor cannot change instantaneously, instead, it has to gradually charge and this gradual charging is governed by the time constant of the RC circuit.

I measured the resistance of R4 to be $10.84k\Omega \pm 0.03$.

To calculate expected delay, we can calculate the time constant, where $R = 10.84k\Omega \pm 0.03$, and $C = 1nF$

$$\tau = RC = (10.84k\Omega \pm 0.03)(1nF) = 10.84\mu\text{s}$$

Note: uncertainty is so small here it is not relevant

The time constant is about equal to my measured delay, suggesting that the delay is simply one time constant. This suggests that the transition voltages of the inverter are low enough, that the capacitor can charge up to enough voltage in one time constant for the inverter to change states, making the delay roughly equal to the time constant.

Next, I tested the reset output of the circuit. This RESET pulse generator provides short pulses (i.e. a pulse train with a with a low duty cycle) that resets the counter after the D-latch grabs the counter's output.

In this circuit, the square wave input is first delayed (to ensure the RESET happens after the data is latched), and then sent to the RESET generator, which creates a pulse (which will be

delayed compared to the square wave, due to the delay circuit). To measure the pulse, I connected the same 5 Hz square wave at the input of the circuit, and I used my channel 2 wires to measure the delayed square wave output at Y6 (see figure 2), and used my channel 1 wires to measure the reset output at Y4.

```
In [11]: InlineImage("resetPulse.png", 11,11, caption = "Figure 8: Reset Pulse ")
```

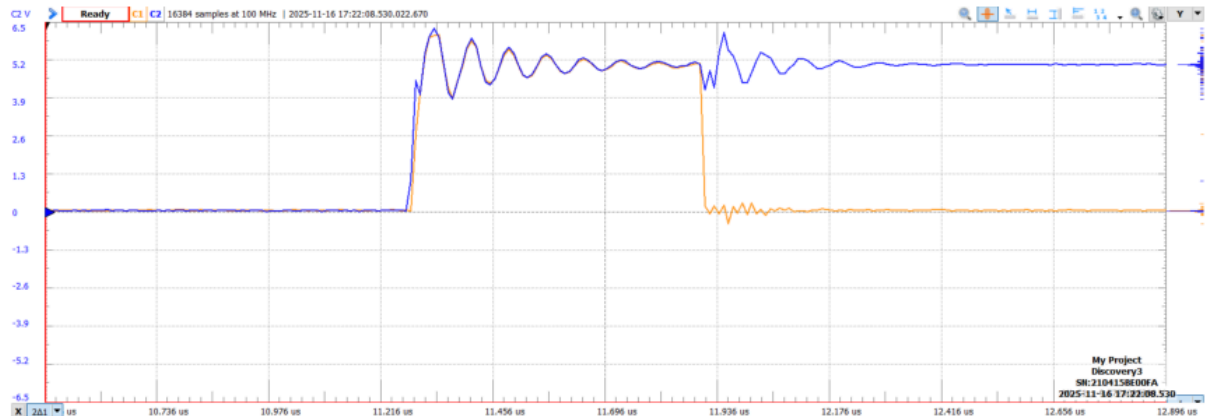


Figure 8: Reset Pulse

Here, we can see the delayed square wave in the blue, and the reset pulse in the yellow (the blue overlaps the yellow, as the reset pulse starts off with almost the same shape as the delayed square wave). What happens here is that the original, undelayed square wave is sent to the D-LATCH, which causes it to latch its input on a rising edge of the square wave. This same square wave is delayed (blue), and on the same rising edge of this delayed wave, a small pulse is generated (yellow). This pulse is sent to the counter, which resets the counter. Since this pulse is generated based on the square wave that is delayed relative to the one that the D-LATCH is receiving, it creates a small delay between when the D-LATCH latches its input, and when the counter is reset.

In this RC network, the capacitor value is much smaller (about 47 pF) than the delay capacitor, so its time constant is much shorter. This means the spike decays very quickly, which produces a short reset pulse.

It should be noted that this reset pulse only happens on a rising edge of the input square pulse, not on the falling edge, as data is only latched on the rising edge, so we want to reset the counters at the same time (after a very small delay). When the input square wave has a rising edge, the voltage across the capacitor cannot change instantly, so current flows through the resistor to ground. This produces a positive voltage spike at the inverter input (because the resistor sees current flowing down to ground). That spike crosses the Schmitt trigger upper threshold and creates the reset pulse.

When the input has a falling edge, the capacitor discharges and current flows from ground through the resistor. This produces a negative voltage spike at the inverter input, but this small negative transient is clamped near 0 V by the inverter's input protection and never

crosses the lower threshold of the Schmitt trigger. Therefore, no reset pulse is generated on the falling edge.

Counter, DLATCH, DAC

The next part of the circuit is shown here.

In [18]: `InlineImage("counterLatch.jpg", 11,11, caption = "Figure 9: Counter, Latch, and DAC`

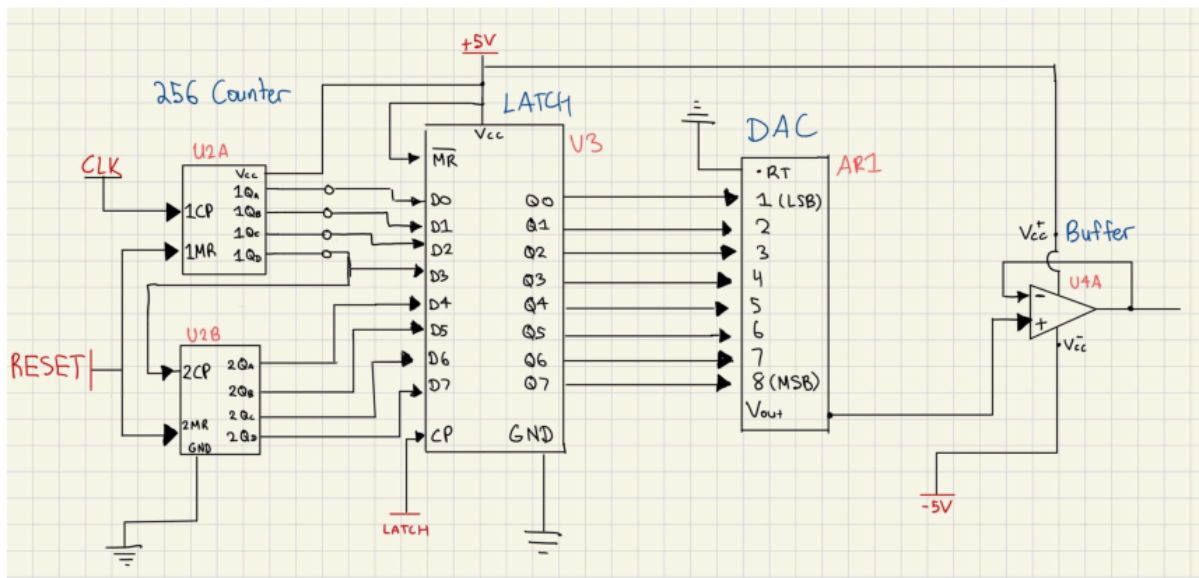


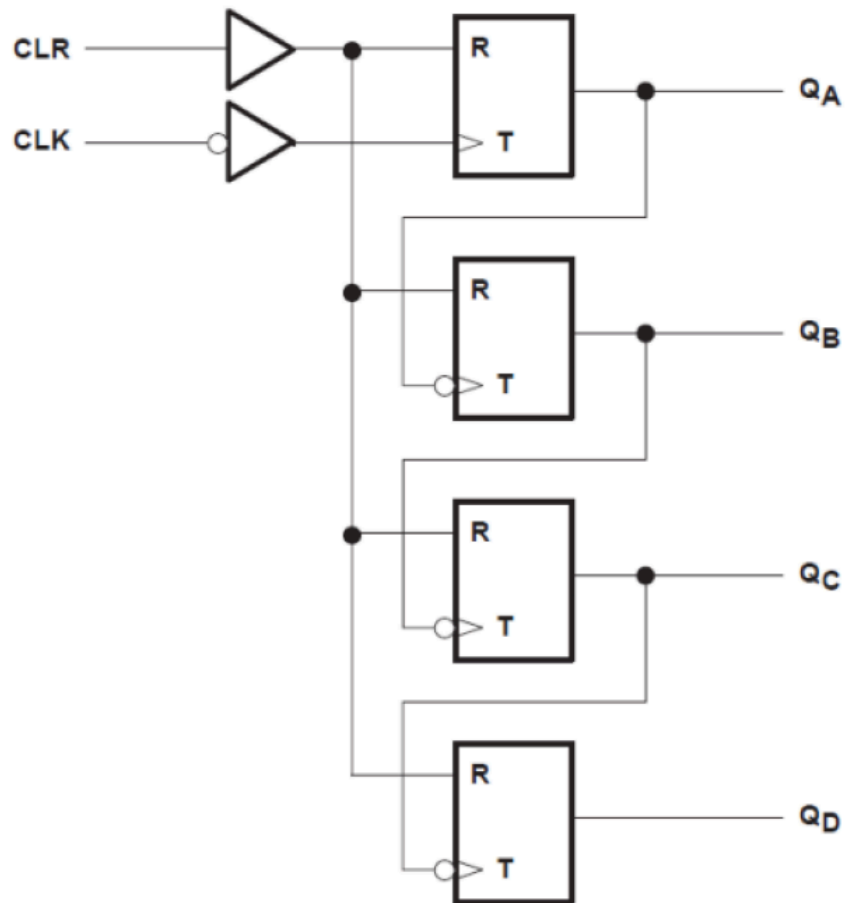
Figure 9: Counter, Latch, and DAC Circuit Diagram

This portion of the circuit counts the number of CLK pulses between RESET pulses and stores the result before converting the binary (or digital) measurement to analog value.

Counter

I had to create a 256 counter. This counter has to count the clock pulses coming into its CLK input (which will be coming from the phototransistor on the motor PCB). A 256 counter can be made by connecting two 16 counters together (U2A and U2B). This was done using the SN74HC393N Dual 4 bit binary counters. Its relevant info is shown below.

In [14]: `InlineImage("clockLogic.PNG", 11,11, caption = "Figure 10: SN74HC393N Logic Diagram`



Logic Diagram, Each Counter (Positive Logic)

Figure 10: SN74HC393N Logic Diagram

In [17]: `InlineImage("counterPinout.PNG", 11,11, caption = "Figure 11: SN74HC393N Pinout Dia`

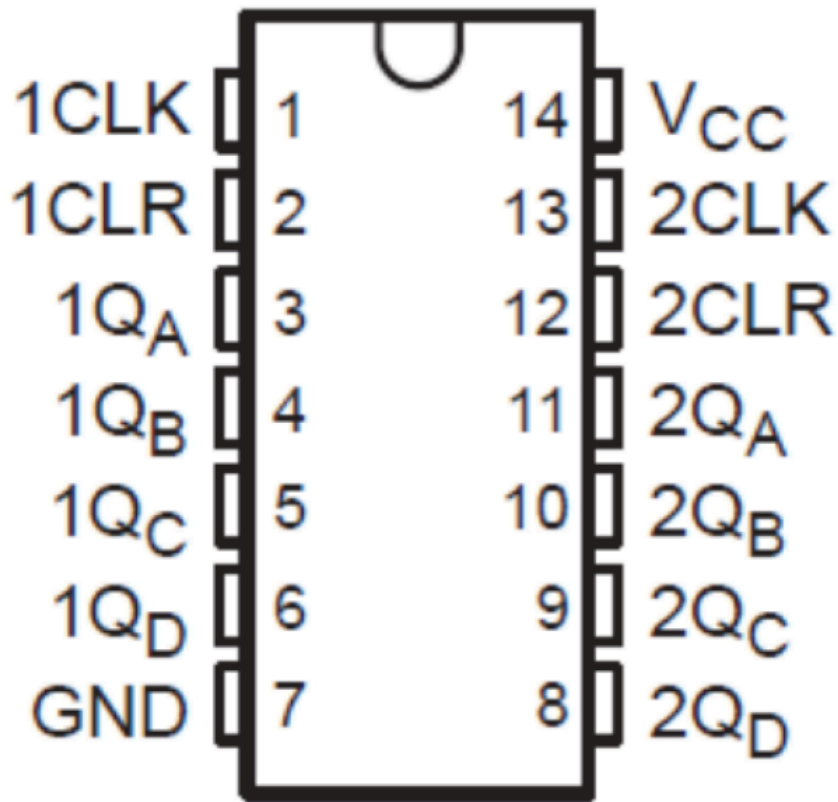


Figure 11: SN74HC393N Pinout Diagram

I started by grounding both the RESETS (which is done just to test the function of the counter, these will later be connected to the reset from the reset generator), and connecting a digital IO pin to 1CLK, to use as my input, and a digital IO pin to 2Qd, to use to measure my output. I connected 1QD (the most significant bit of U2A, seen in figure 9) and 2CLK (the input of U2B) together to create the 256 counter (this is like connecting two 16 counters to each other).

In [18]: `InlineImage("256counterphoto.jpg", 11,11, caption = "Figure 12: 256 Counter Photo`

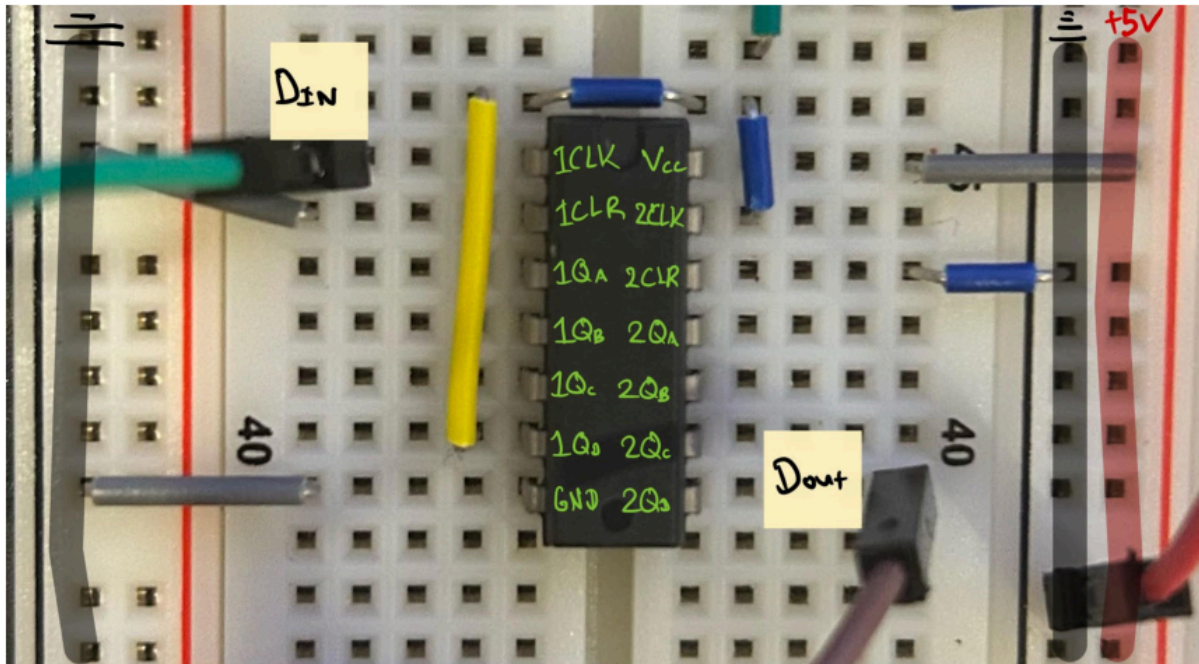


Figure 12: 256 Counter Photo

At the input, I used my AD3 to generate a low frequency pulse train (5 Hz), and verified the function of the counter. I connected DIO pins to all 8 outputs, and created a BUS of all 8 bits, to see the decimal count. 2Qd is the most significant bit, and 1Qa is the least significant.

In [11]: `InlineImage("256counteroutput.png", 13,13, caption = "Figure 14: 256 Counter output`

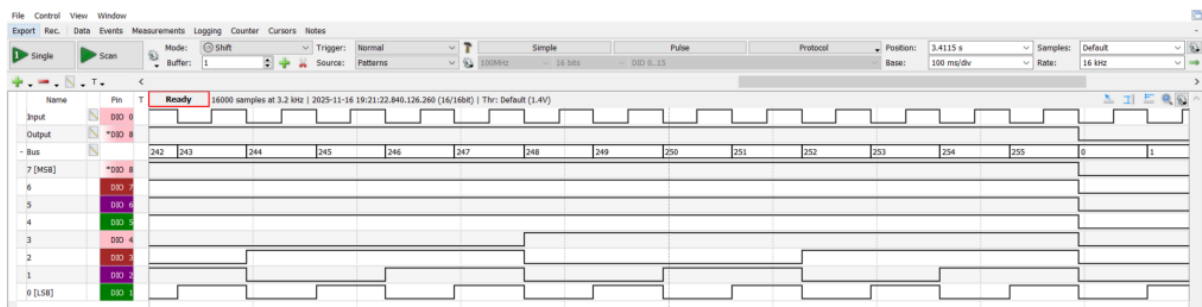


Figure 14: 256 Counter output

Using the logic analyzer, the 8-bit counter was observed counting from 0 to 255. After reaching 255, the next clock pulse caused the counter to overflow and return to 0. Therefore, the maximum representable value is 255, and values above this cannot be stored, instead the counter wraps back to 0. I also checked the reset functionality with a push-button switch, which I did by manually wiring a digital IO pin to 1MR and 2MR. This let me to manually trigger the reset, so instead of resetting at 255, it would reset whenever I want it to. This verified that the reset function of the counter is working correctly, because later, the reset will come from a pulse from the reset generator.

LATCH

For the next part of the circuit, I set up the D-LATCH. The counter outputs are connected directly to the latch, which is responsible for latching the data from the counter when the square wave from the LATCH output from the inverter hits a rising edge (which happens every 0.2s, since the frequency of the input square wave is 5 Hz).

The component used is a 74HCT273, with relevant info shown here.

```
In [17]: InlineImage("DLatchPinout.PNG", 11,11, caption = "Figure 15: 74HCT273 Pinout")
```

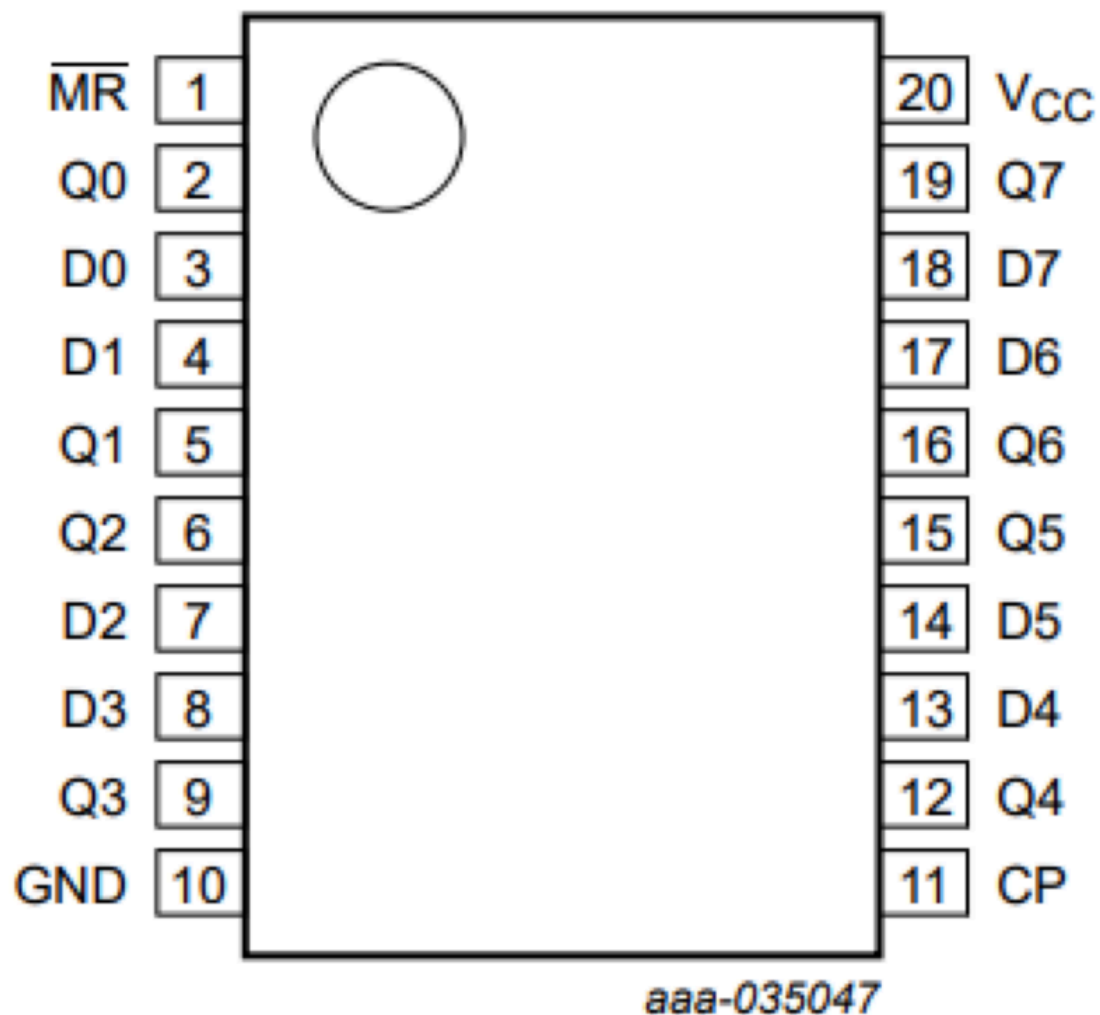


Figure 15: 74HCT273 Pinout

```
In [16]: InlineImage("DLatchFunctionalTable.PNG", 14,14, caption = "Figure 16: 74HCT273 Func
```

*H = HIGH voltage level; h = HIGH voltage level one set-up time prior to the LOW-to-HIGH clock transition;
 L = LOW voltage level; l = LOW voltage level one set-up time prior to the LOW-to-HIGH clock transition;
 X = don't care; ↑ = LOW-to-HIGH clock transition.*

Operating modes	Inputs			Outputs
	$\overline{\text{MR}}$	CP	Dn	Qn
reset (clear)	L	X	X	L
load "1"	H	↑	h	H
load "0"	H	↑	l	L

Figure 16: 74HCT273 Function Table

In [14]: `InlineImage("dlatchLogicDiagram.PNG", 12,12, caption = "Figure 17: 74HCT273 Logic D`

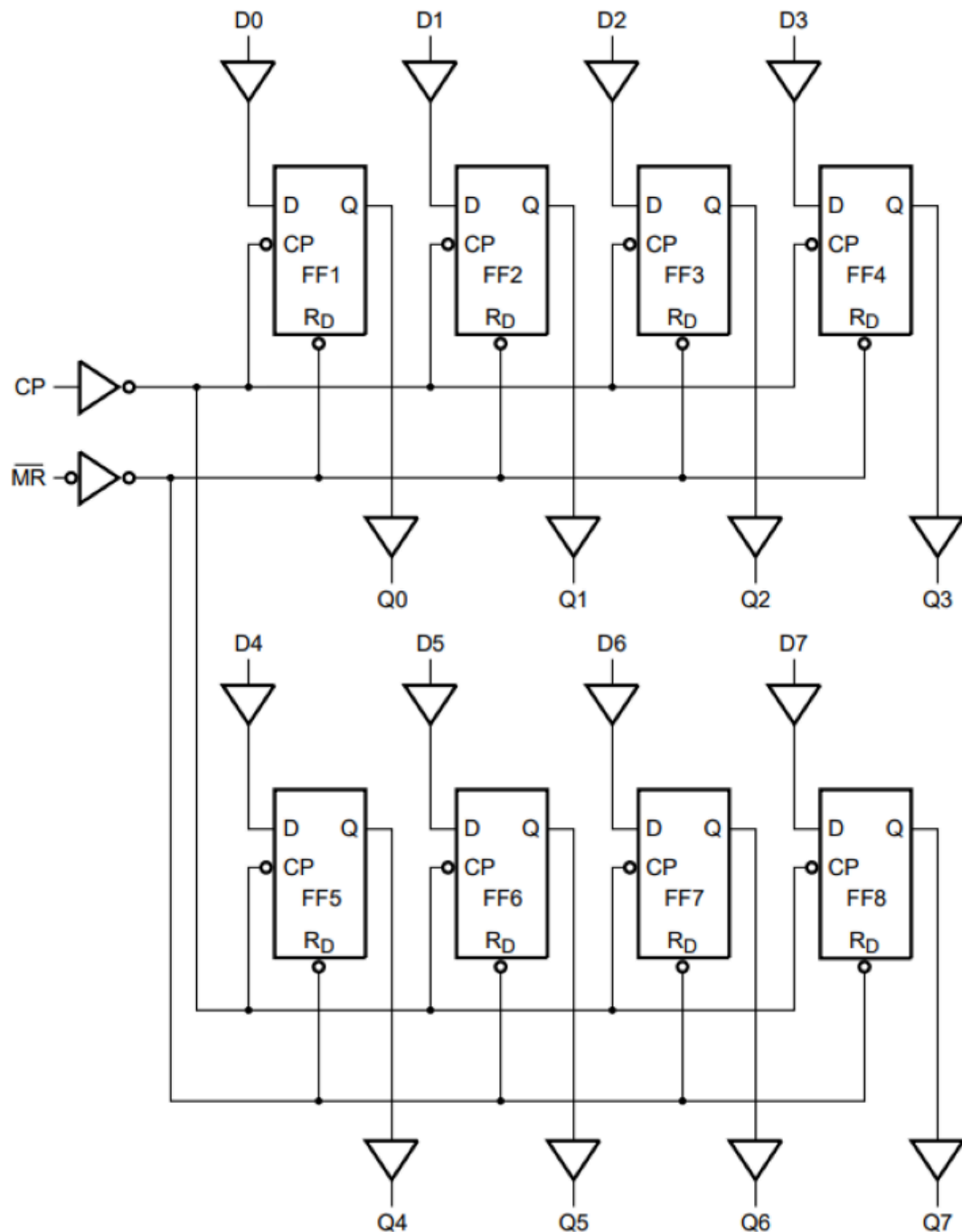


Figure 17: 74HCT273 Logic Diagram

I connected the 8 bit outputs of the counter to the latch, and the LATCH output from the delay and reset circuit (long red wire) to the CP pin of the latch (which controls when it takes a new "snapshot" of data), shown below.

In [15]: `InlineImage("LatchCircuitPhoto.jpg", 11,11, caption = "Figure 18: Latch Photo ")`

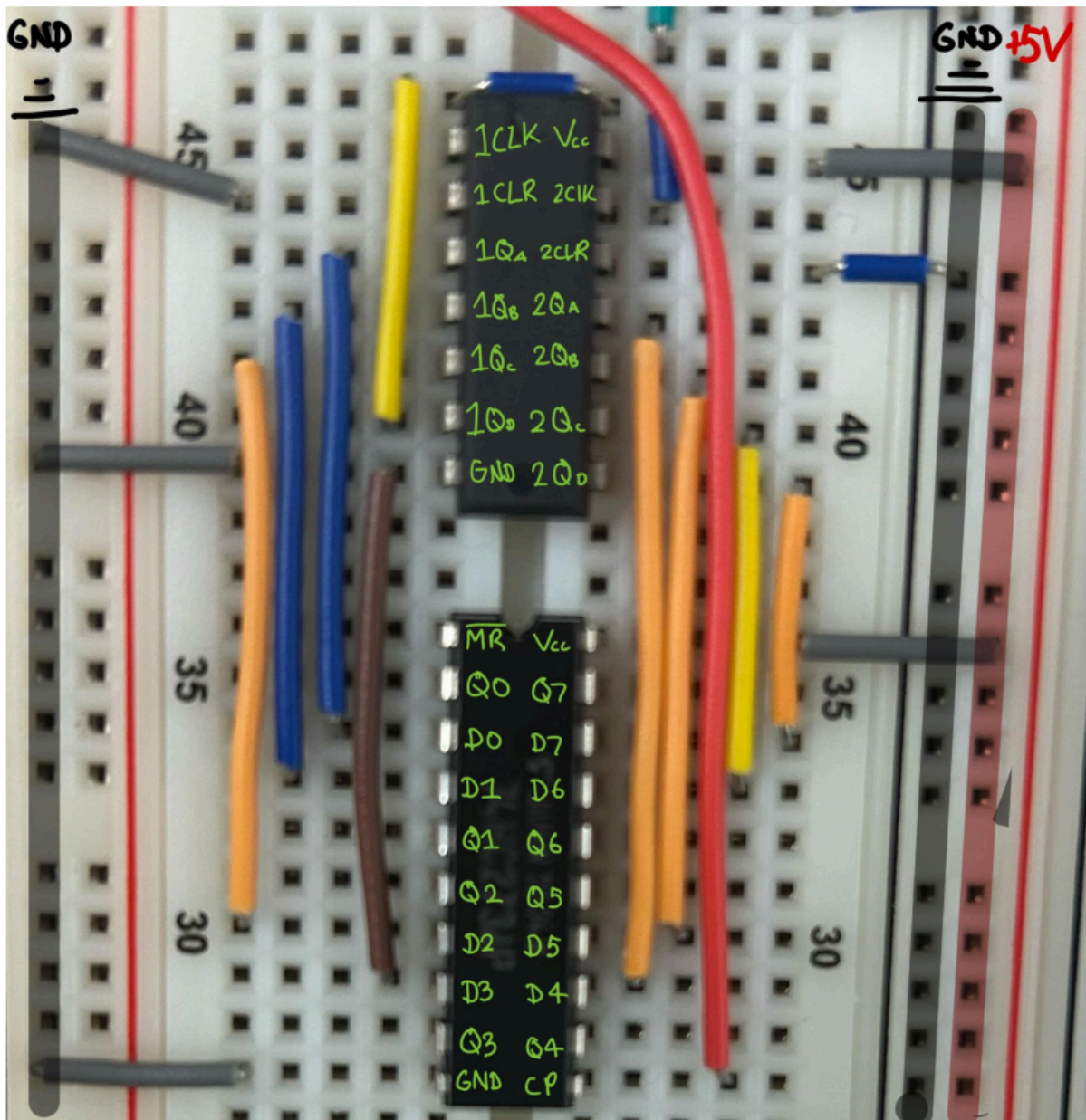


Figure 18: Latch Photo

Although I forgot to show this in the picture, I also connected the latch reset to 5V, and then I also connected the RESET output from the delay and reset circuit to my counter, instead of having the 2 RESETS on the counter grounded. This allows me to test the function of my circuit all together, and see if everything works as expected up to the latch.

We can determine what the Latched value should be based on the frequency of the input into the inverter (delay and reset circuit), and the frequency of the input to the counter. Let's call the inverter input frequency f_I and the counter input frequency f_C .

The count of the counter increases by 1 every $\frac{1}{f_C} = T_C$ seconds. The counter is reset every $\frac{1}{f_I} = T_I$ seconds. This means that before the counter is reset, it counts $\frac{1}{T_C} * T_I$ times, which is

$\frac{T_I}{T_C}$, or in other words

$$\text{Latch Value} = \frac{f_C}{f_I}$$

I inputted a 5 Hz square wave oscillating between logic level HI and LO (5 V and 0 V) into my delay and reset circuit , and then gave a 500 Hz test clock pulse to the input of the counter using the patterns tool on my AD3 (in the actual circuit, the clock pulse is coming from the motor itself, this manual input is just for testing). This means the Latched value we should expect to see is 100.

In order to measure the latched value, I connected 8 DIO pins to the outputs of the latch, and created a BUS on the logic tool of my AD3.

TS

At first, I was not getting a stable latch value. I know that since my test clock pulse is at a stable frequency, the latched value cannot change over time. I checked both my input waves to make sure they had the correct settings, which they did. Next, I checked that all the outputs from the counter were connected to the correct corresponding latch inputs. From this, I discovered that I had wired one of the inputs on the latch incorrectly (instead of being wired from the clock, it was wired to ground). However, I realized that if the input was wired to ground, the latch value should still have been stable (although it would have been the incorrect value), so I knew something else was still wrong. I then checked the reset pins on the counter, and realized they had not propoerly been connected to the reset pulse from the inverter, meaning the counter was not resetting correctly (since the reset pin was floating, it may have been resetting at random times), which caused me to see incorrect values.

After fixing these issues, I saw the correct latched value of 100. I then changed f_C to 1000 hz, and got a latched value of 200, which confirmed my latch was working correctly.

In [16]: `InlineImage("latchOutput.PNG", 16,16, caption = "Figure 19: LATCH Output")`

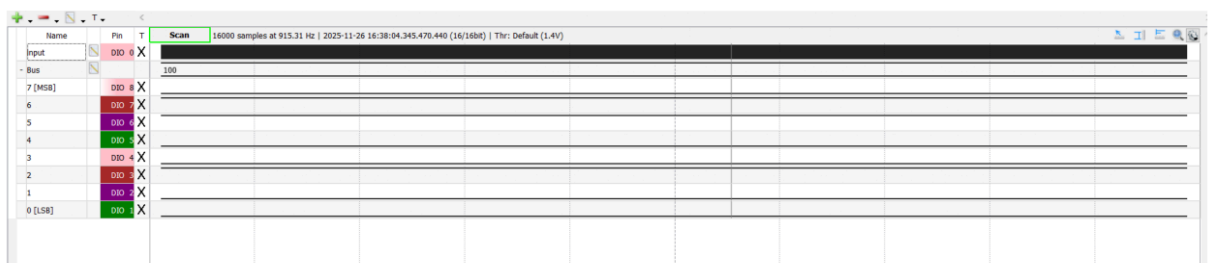


Figure 19: LATCH Output

DAC

Knowing that my LATCH works, the next task was to connect the LATCH outputs to the DAC, which is what will convert the digital signal from the LATCH into an analog signal (refer back

to Figure 9 for circuit diagram).

This is a 4610X-R2R-103LF resistor ladder, with the pinout diagram shown here.

In [17]: `InlineImage("R2RPins.PNG", 11,11, caption = "Figure 20: 4610X-R2R-103LF PINOUT ")`

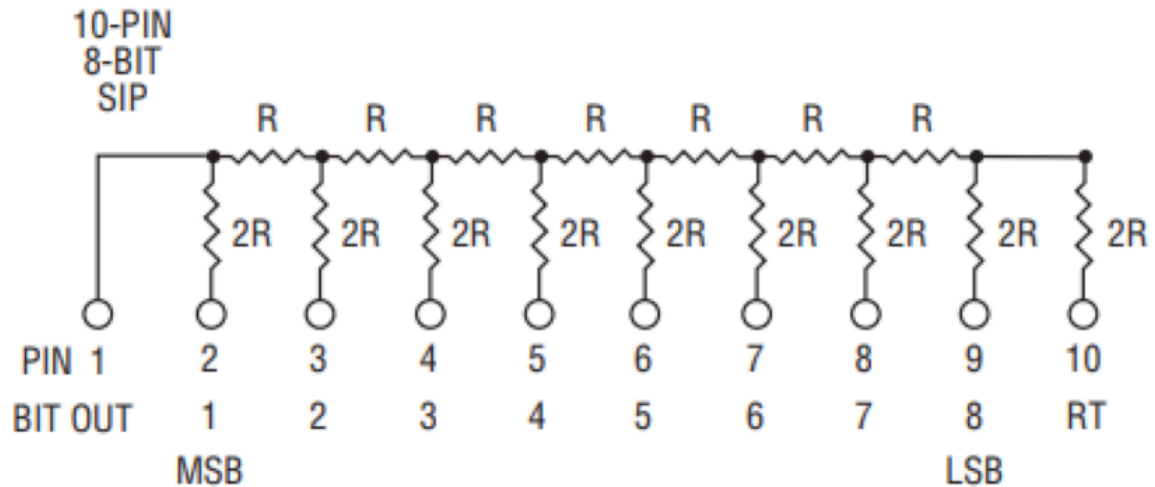


Figure 20: 4610X-R2R-103LF PINOUT

I set this up as seen in figure 9, connecting each output from the LATCH to the DAC in order, with Q0 being the least significant bit, and Q7 being the most significant, and additionally also grounding the RT (terminating resistor), as shown in the circuit diagram.

In [18]: `InlineImage("dacCircuitPhoto.jpg", 11,11, caption = "Figure 21: DAC Photo ")`

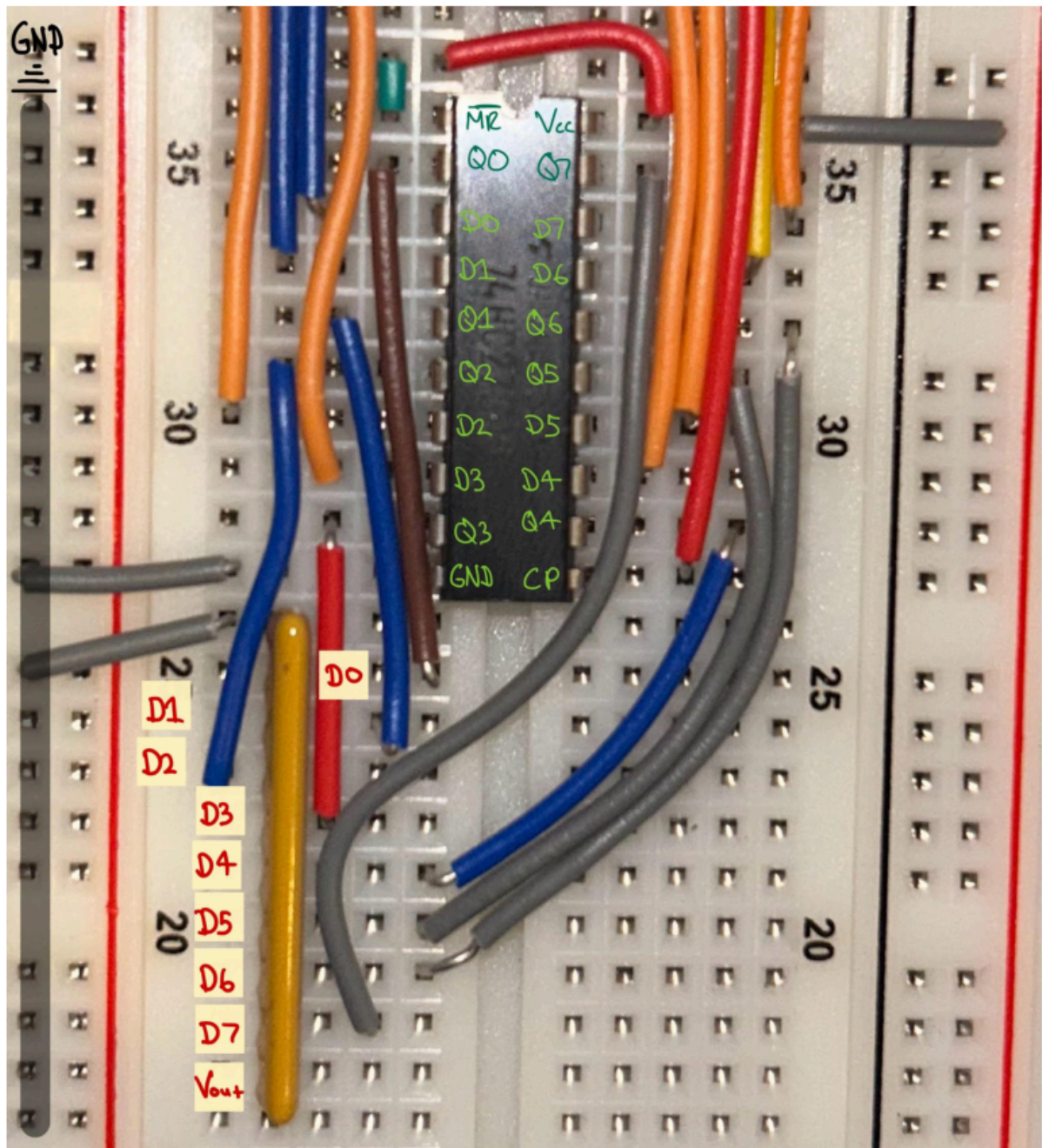


Figure 21: DAC Photo

Before testing the output of the DAC, I connected it to an UA741 op-amp, which is used as a buffer (seen as U4A in Figure 9). This DAC has a very high output resistance, so the job of the buffer is to keep the same voltage output, but remove this output resistance, so it is not seen by the rest of the circuit. This is the pinout diagram for the op-amp, and a photo of the circuit. Unlike the other components where we only needed +5V for the voltage supply, this also requires a -5V supply.

In [20]: `InlineImage("opAmpPinout.PNG", 11,11, caption = "Figure 21: UA741 Pinout")`

uA741C D, P, or PS Package 8-Pin SOIC, PDIP, SO Top View

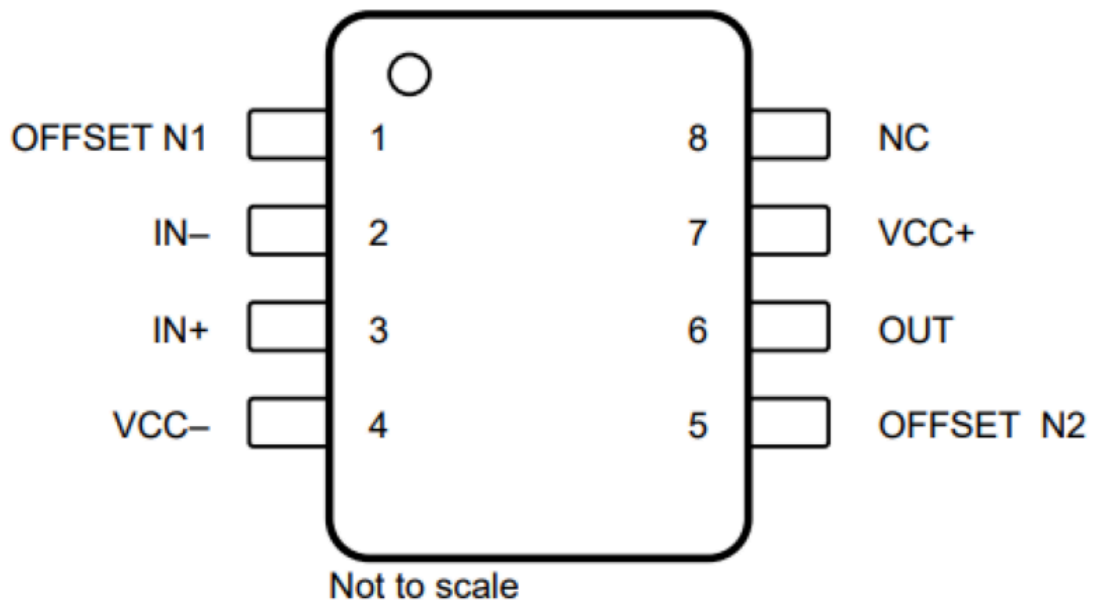


Figure 21: UA741 Pinout

I connected the output of the DAC to IN+ on the OP amp, and set up the negative feedback between IN- and OUT.

In [21]: `InlineImage("bufferPhoto.jpg", 11,11, caption = "Figure 22: Buffer Photo")`

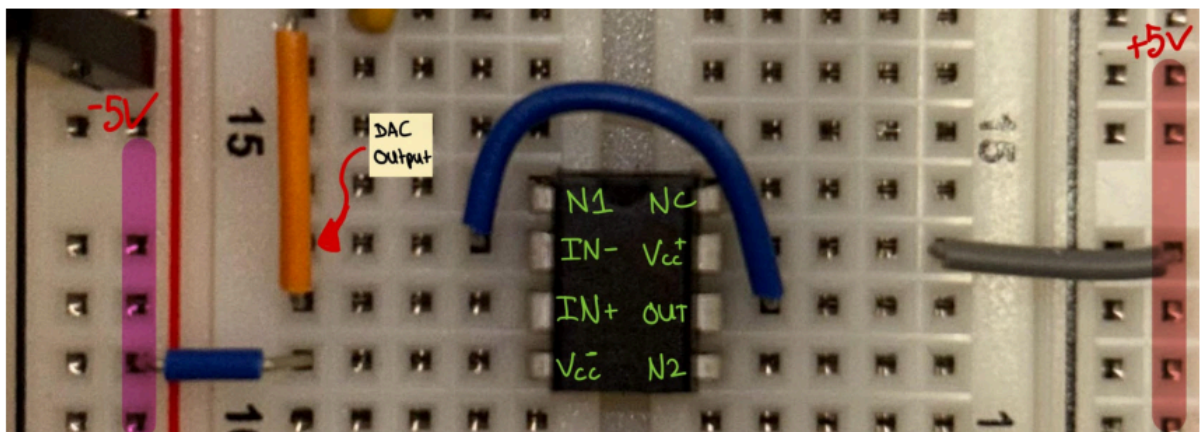


Figure 22: Buffer Photo

We know that the output at the DAC (and hence the buffer, since they have the same output) must scale linearly with the output at the latch. So, if I test different latch outputs in linear increments, the output of the op amp should also increment linearly.

I already know a formula for the value stored in the latch, which depends on the inputs at the inverter and the counter,

$$\text{Latch Value} = \frac{f_C}{f_I}$$

I connected my scope to the output of the buffer, and keeping the input at the inverter at 5 Hz, I increased the frequency of the input going into the counter in increments of 250 Hz. I started with 500 Hz (which corresponds to the a latched value of 100). Increasing the counter frequency in increments of 250 hz corresponds to increasing the latched value by 50, meaning the analog voltage should increase linearly.

In the scope feature of my AD3, I was able to measure an output, and add it as a reference, so that I could change my input parameter and then see another output, and repeat this process. This is how I got the outputs at the buffer for 500 Hz, 750 Hz, 1000 Hz, and 1250 Hz for the inverter frequencies. The top line is the output for 1250 Hz, the one below that is for 1000 Hz, and so on. Since these outputs are linearly spaced, we have verified that the output of the buffer scales linearly with the latched value, which verifies that the DAC is working correctly.

In [22]: `InlineImage("opAMPoutput.PNG", 11,11, caption = "Figure 23: Buffer Output Measureme`

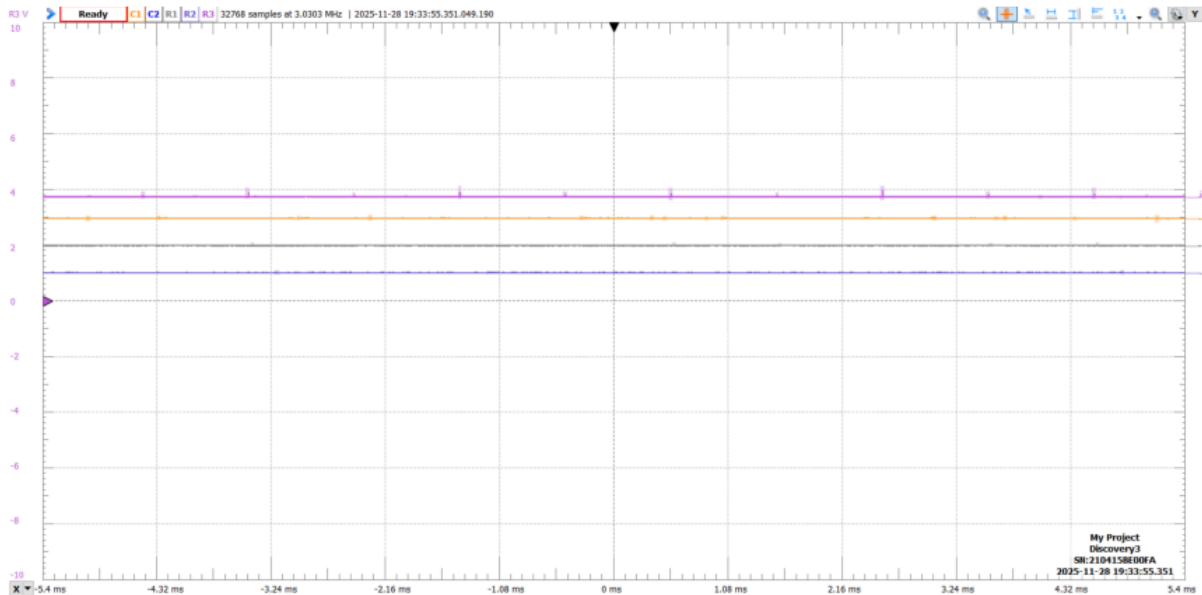


Figure 23: Buffer Output Measurements

Error signal amplifier

The next part of the circuit is the error signal amplifier, which essentially provides the feedback in the circuit by integrating the error between the set voltage (the desired motor speed) from a 10kΩ variable resistor, and the current motor speed (which will be stored in

the latch), coming from the buffer. The output of this operation is then used to control the current through the motor via power transistor.

This uses the same UA741 op-amp as the buffer (same pinout diagram).

In [28]: `InlineImage("errorAMP.jpg", 11,11, caption = "Figure 24: Error Signal Amplifier Cir`

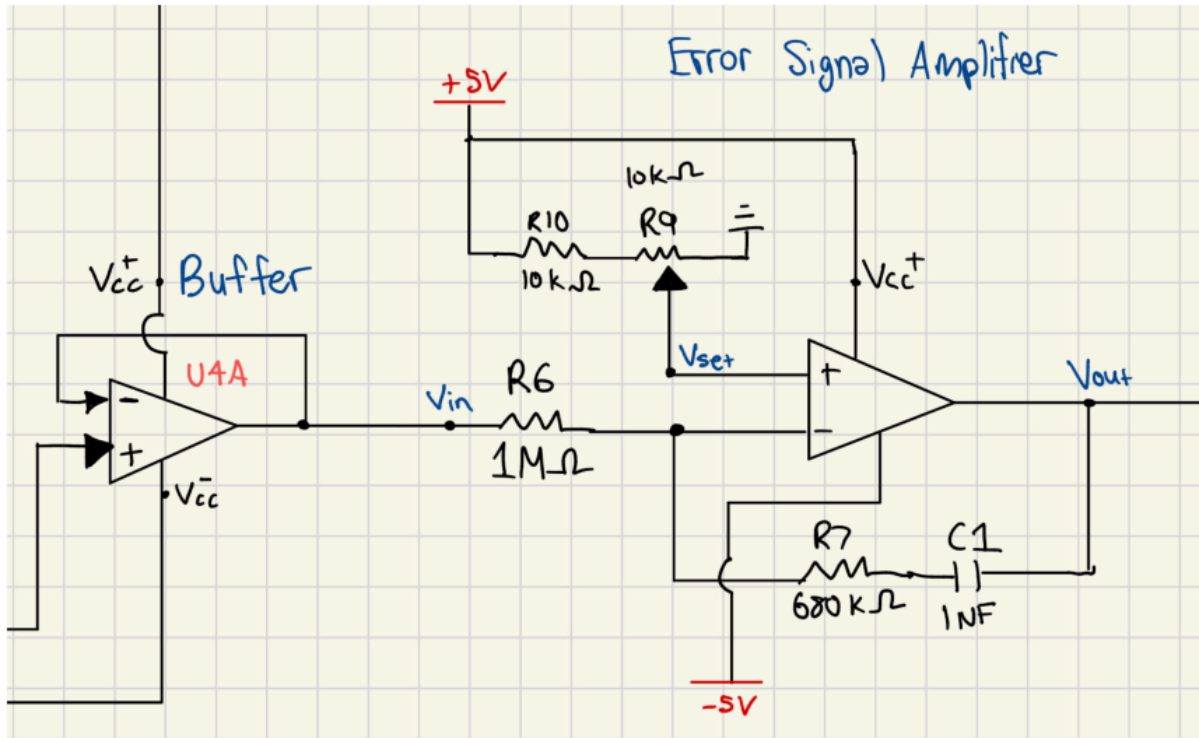


Figure 24: Error Signal Amplifier Circuit Diagram

We can derive an expression of the output voltage of the op-amp as a function of the input voltage V_{in} coming from the buffer (seen in the figure above).

First, we will call the output voltage set by the potentiometer (R9) as V_{set} . We know the potentiometer can at most, act as a 0Ω resistor, in which case, V_{set} is $0V$. At most, the POT can act as a $10k\Omega$ resistor, in which case it acts as a voltage divider with the other $10k\Omega$ resistor to its left, at which point V_{set} is $2.5V$ (half of the $5V$ input). Therefore, we know that the voltage set by the potentiometer (which is the voltage at the positive input of the op-amp) has a range of $0V \leq V_{set} \leq 2.5V$.

We can derive an expression for $V_{out}(t)$ at the output of the error signal amplifier

Knowing that $V_+ = V_-$ for an op amp, we know V_- is also equal to V_{set} . We can denote I as the current going through $R6$, which is the same current that goes through $C1$ and $R7$ (since no current can go into the op amp).

$$I = \frac{V_{in} - V_{set}}{R_6}$$

We can also write the current through C1 and R7 in terms of their voltages, which allows us to isolate for their voltages.

$$I = C \frac{dV_c}{dt} = \frac{V_{R7}}{R_7}$$

$$V_c = \int_0^t \frac{I}{C} d\tau$$

$$V_{R7} = IR_7$$

We know the voltage across the two components C1 and R7 must be equal to:

$$V_{set} - V_{out} = V_{R7} + V_c$$

Isolating for V_{out}

$$V_{out} = V_{set} - V_{R7} - V_c$$

Plugging in expressions for voltages from above

$$V_{out} = V_{set} - IR_7 - \int_0^t \frac{I}{C} d\tau$$

$$V_{out} = V_{set} - \frac{R_7(V_{in}(t) - V_{set})}{R_6} - \frac{1}{R_7 C} \int_0^t (V_{in}(\tau) - V_{set}) d\tau$$

Plugging in component values

$$V_{out}(t) = 1.68 V_{set} - 0.68 V_{in}(t) - \int_0^t (V_{in}(\tau) - V_{set}) d\tau$$

If we take the time derivative of this (keeping in mind that V_{set} has no time dependence, so its derivative goes to 0), we get

$$\frac{dV_{out}}{dt} = -0.68 \frac{dV_{in}}{dt} - (V_{in}(t) - V_{set})$$

$$\frac{dV_{out}}{dt} = V_{set} - V_{in}(t) - 0.68 \frac{dV_{in}}{dt}$$

Knowing this, I then built the error signal amplifier, seen here.

In [29]: `InlineImage("errorAmpPhoto.jpg", 11,11, caption = "Figure 25: Error Signal Amplifier")`

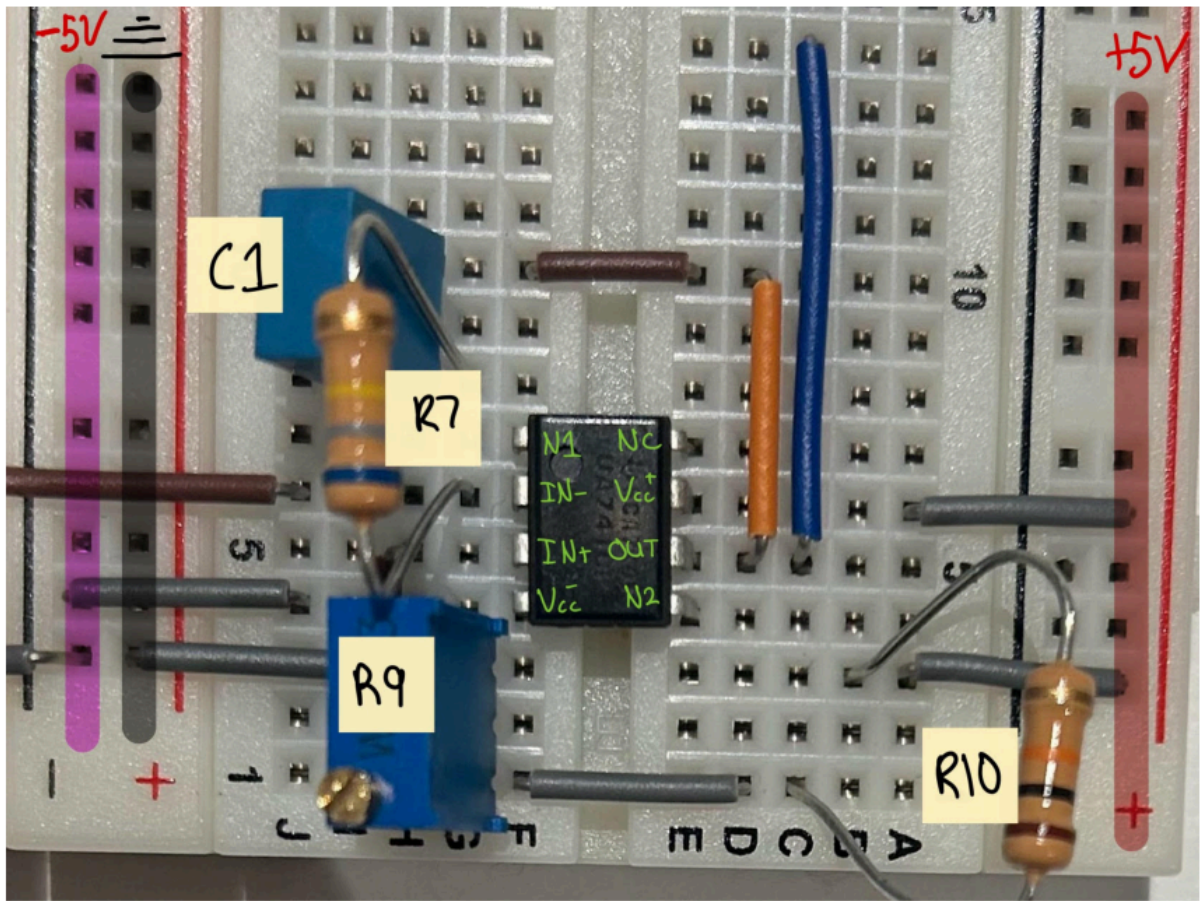


Figure 25: Error Signal Amplifier Photo

R6 is further off to the left, at the buffer, and the input will go through that before going into IN-.

Now I had to actually test the error amplifier. The expression for V_{out} is somewhat complicated involving an integral, however, we can look at the expression for the time derivative of the output, that was derived earlier.

$$\frac{dV_{out}}{dt} = V_{set} - V_{in}(t) - 0.68 \frac{dV_{in}}{dt}$$

This means in order to test the error amplifier, I can use a square wave input for V_{in} . In this case, V_{in} will oscillate between 2 values, which means $\frac{dV_{out}}{dt}$ will also oscillate between two values. $\frac{dV_{in}}{dt}$ will be 0 for a square wave, atleast for every moment other than at the instant V_{in} switches between its two values.

This means that the slope of the output can be written as $\frac{dV_{out}}{dt} = V_{set} - V_{in}$

If we use a square wave for V_{in} , the slope of the output will oscillate, because the value of V_{in} is oscillating.

If V_{in} oscillates between V and $-V$, the slope of V_{out} will oscillate between $V_{set} - V$ and $V_{set} + V$. This will create a shape similar to a triangle wave.

TS

I set up the manual input in front of R6 (not seen in the photo above, it is connected to IN-), and disconnected the error amplifier from the rest of the circuit. Initially, I did not think much of the settings I was using, and I set V_{set} to 2V (by measuring the output directly at the potentiometer), and set V_{in} to oscillate between 0 and 5V (so the slope would oscillate from 2V and -3V) at 2 Hz. However I found that the op-amp kept clipping, and I was not seeing the slopes I was expecting, instead just straight lines (0 slope). At first, I thought I had wired something incorrectly, so I checked the wiring. However, I realized that this is expected of this circuit. The slope of 2V and -3V were too high, it was causing the voltage to go too high for the opamp.

So, to counteract this, I tried different settings for V_{set} and V_{in} until I found something that did not clip. I lowered the frequency of the square wave to 1 Hz (less time for voltage to increase before slope switches), and made it oscillate between 1.5V and 2.5V, and I set V_{set} to 2V. With these settings, we know that the slope of the wave will oscillate between $\pm 0.5V/s$

The output is shown here

In [33]: `InlineImage("errorAmplifierOutput.png", 11,11, caption = "Figure 26: Error Amplifie`

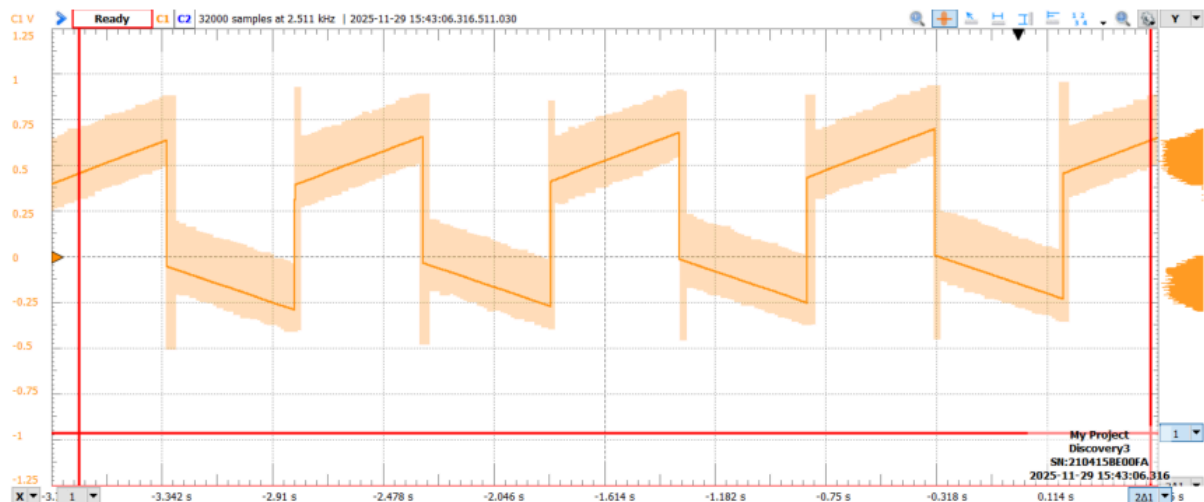


Figure 26: Error Amplifier Output

Using the cursor tools, I manually calculated the slopes that the output is switching between, and I got $\pm 0.5V \pm 0.05$, which is what was expected (error found using formula 1).

I can also observe that this output isn't an exact triangle wave, as I might have expected before. Instead, it does alternate slopes like a triangle wave would, but it has vertical lines in between these alternate slopes. I realized that this happens because of the fact that we are using a square wave as the input, and V_{out} is dependent on the slope of the square wave,

which I assumed to be 0 (seen in the formula for $\frac{dV_{out}}{dt}$). However, in the moment that the square wave switches from high to low, or low to high, it has an "infinite" slope (or at least an extremely large one). So because of this, $\frac{dV_{in}}{dt}$ is momentarily very large, causing $\frac{dV_{out}}{dt}$ to be momentarily very large, which is what creates those vertical jumps in between the different pieces in the output.

Motor Control with BJT

Next, I had to make the connections to the phototransistor PCB adapter board. The circuit diagram for this PCB is shown here.

In [34]: `InlineImage("PCBDiagra.jpg", 11,11, caption = "Figure 27: PCB Connections Diagram "`

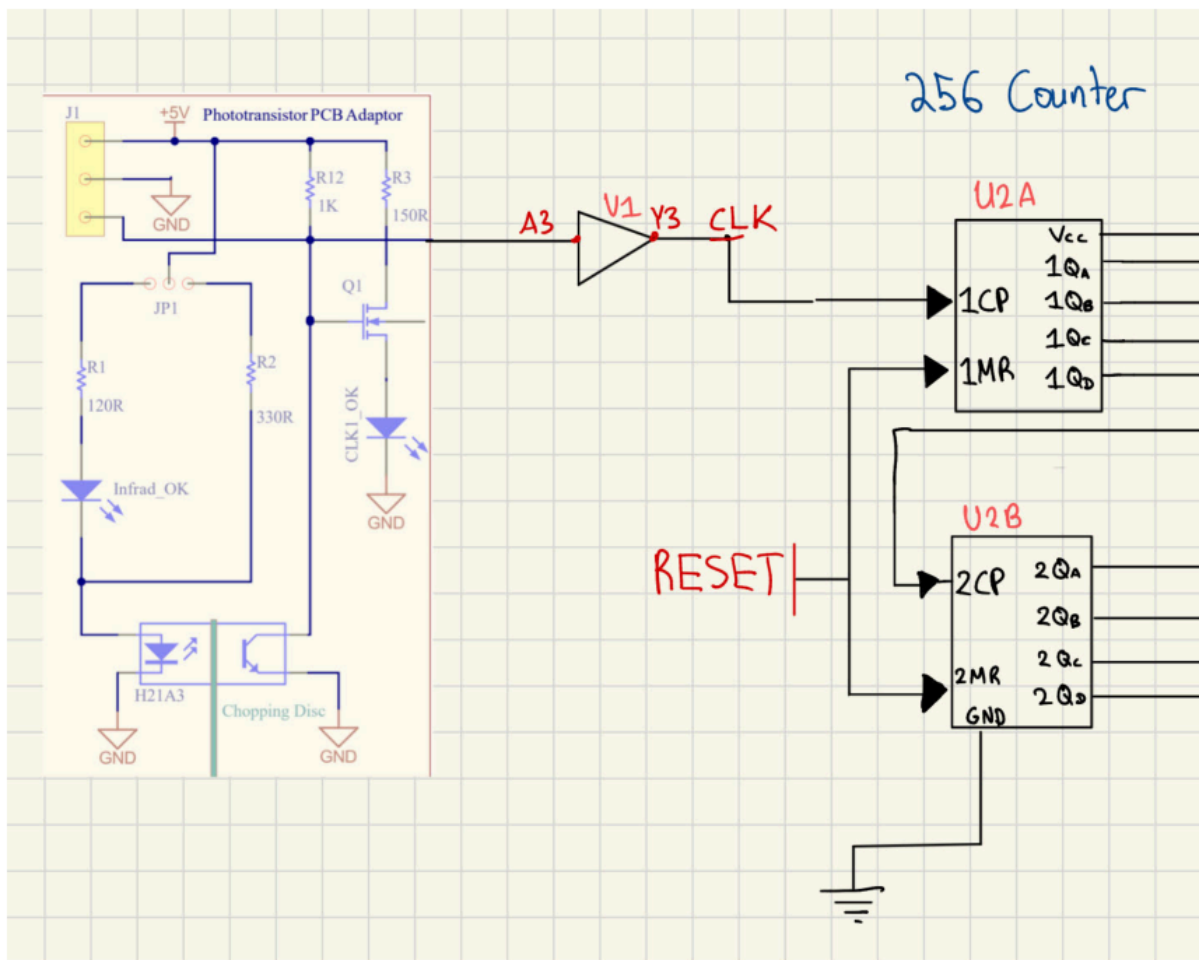


Figure 27: PCB Connections Diagram

Via the jumper J1, I connected the 5V pin to the power rail, the GND pin to the GND Rail, and as shown in the circuit diagram, I connected the clock pin into the one remaining inverter I had left (this was not used before, so its input was grounded, this corresponds to pin A3 seen in figure 6). I connected the output of this last inverter (Y3) into the clock input

of the counter (so I removed the test DIO pin that I was using previously, since we have our actual input now).

The motor is connected to a disk with 10 slits in it. When the motor is turned on the disk rotates, and when a slit passes by the LED some light falls on a phototransistor, causing it to conduct thereby generating a clock pulse. This is the pulse that we want to use as the input for our counter (and later have this count be latched and converted into an analog value), which is why the clock output from the PCB is wired to the counter.

```
In [35]: InlineImage("pcbConnections.jpg", 11,11, caption = "Figure 28: PCB Connections Phot
```

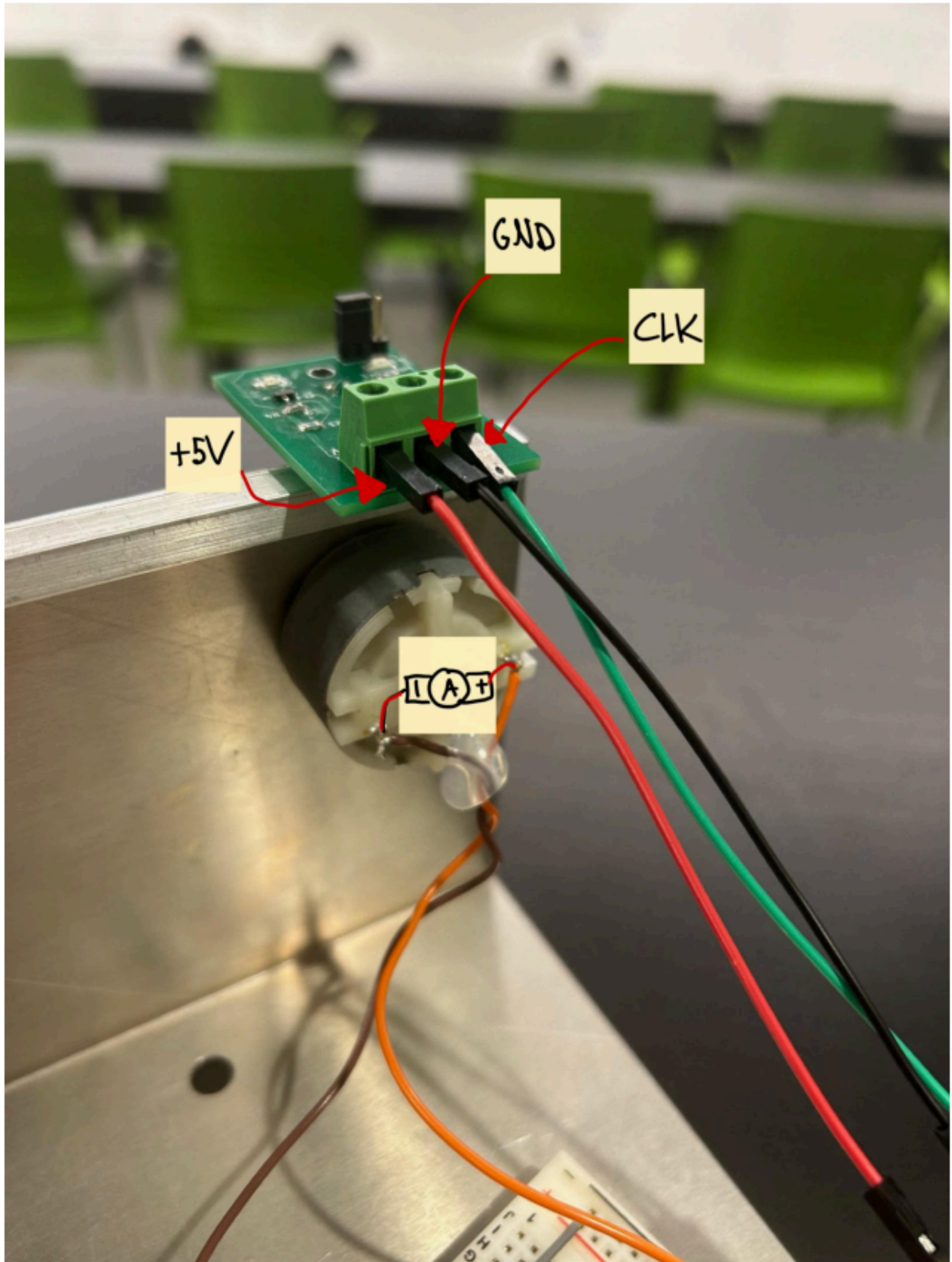


Figure 28: PCB Connections Photo

Once I connected the PCB to ground and 5V, I put my scope to the output of the inverted clock pulse, and I manually rotated the wheel, to make sure I was getting a proper square pulse, confirming my photo transistor is working.

```
In [4]: InlineImage("clockOutputSquare.png", 11,11, caption = "Figure 29: Square CLK pulses
```

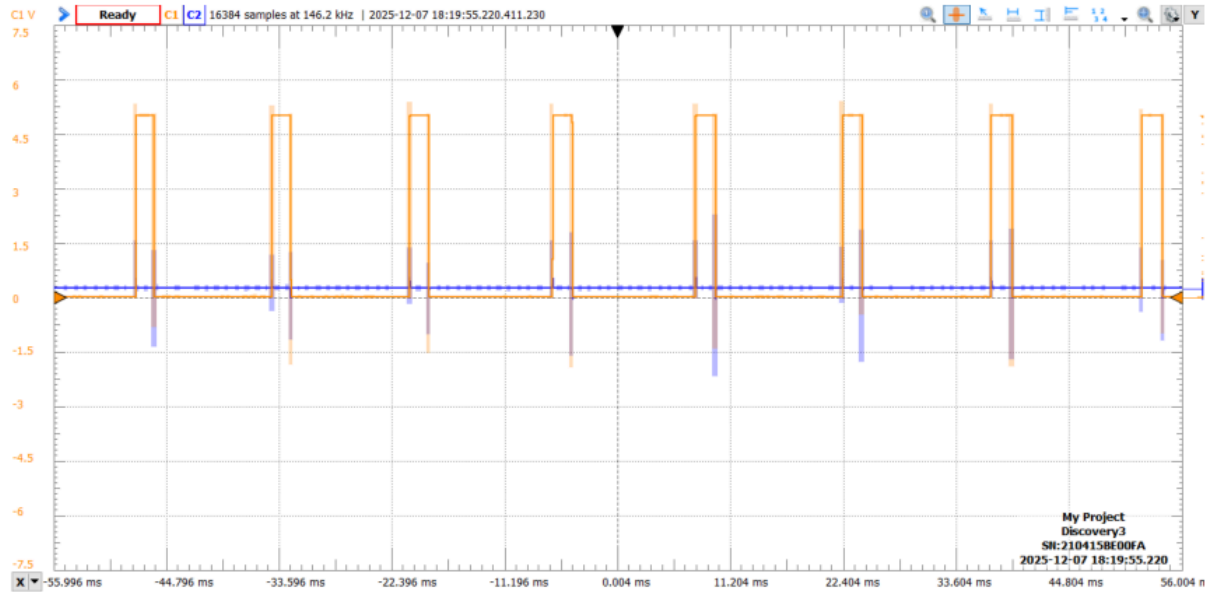


Figure 29: Square CLK pulses from Disk Rotation

Similarly, I connected my DIO pins to the output of the latch, and manually rotated the wheel at different speeds, to confirm that I was getting different latched valued depending on the speed of the wheel.

This is when I moved it very slowly by hand, I got a latched value of 3.

```
In [13]: InlineImage("slowMotion.png", 11,11, caption = "Figure 30: Latch Value at low Speed
```

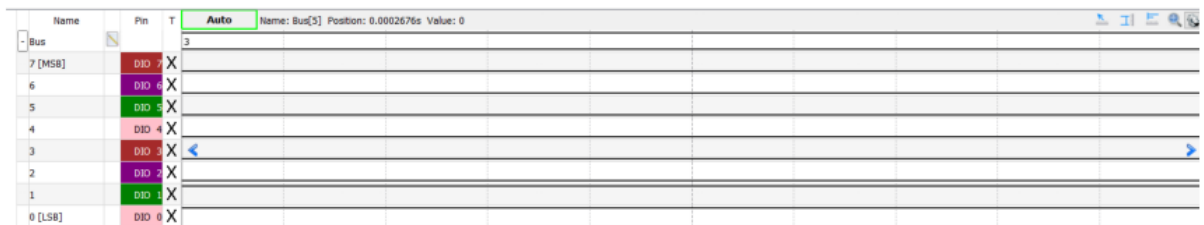


Figure 30: Latch Value at low Speed

I connected the motor wires directly to power (one to 5V, one to GND), so it was essentially moving at its maximum possible speed. I got a latched value of 155, confirming that the latch value changes with the speed of the wheel. I also tried to move it manually, but at different speeds, and I was able to confirm that the latched value changed based on the speed I moved the wheel.

```
In [12]: InlineImage("motorFast..png", 11,11, caption = "Figure 31: Latch Value at Fast Speed
```

Name	Pin	T	Auto	16384 samples at 1.5873 MHz 2025-12-07 18:42:01.540.326.230 (16/16bit) Thr: Default (1.4V)
Bus			155	
7 [MSB]	DIO 7	X		
6	DIO 6	X		
5	DIO 5	X		
4	DIO 4	X		
3	DIO 3	X		
2	DIO 2	X		
1	DIO 1	X		

Figure 31: Latch Value at Fast Speed

Next, I had to connect the motor itself to the circuit.

In [11]: `InlineImage("motorDiagram.jpg", 11,11, caption = "Figure 32: Motor Connections Circ`

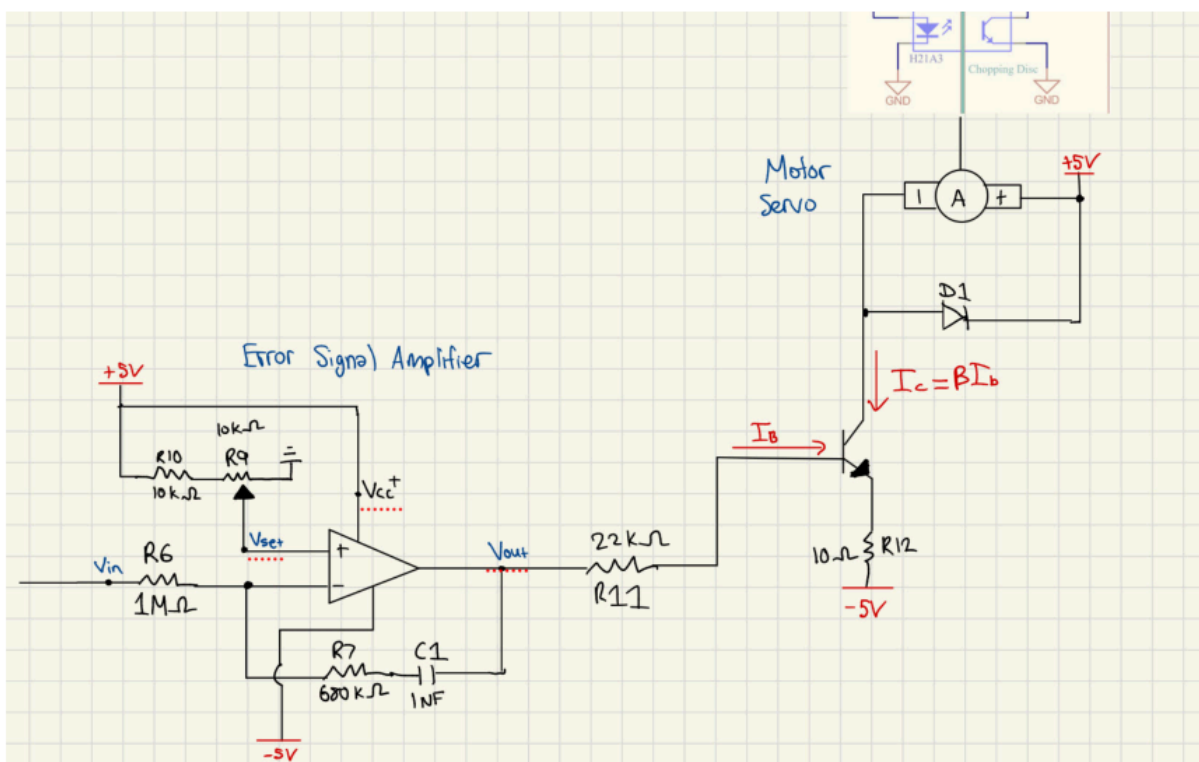


Figure 32: Motor Connections Circuit Diagram

Note that the part above the motor Servo is the pcb circuit.

The positive connection of the motor is connected directly to the 5V power rail, and the negative connection is connected to the part of the circuit with the diode and BJT.

The error amplifier will produce an output voltage according to how far the current motor speed is from the set speed. This voltage creates current through R11, however, this current is not strong enough to actually drive the motor, which needs a relatively high current to spin. The BJT acts as a current amplifier, where the small current through the base creates a large current at the collector (which flows from +5V through the motor), enough to drive the motor at the desired speed. Because $I_c = \beta I_b$ the motor current (and hence motor speed)

increases as the base current increases. As the error voltage increases, the base current increases, Q1 conducts more, and the motor current (and hence speed) increases until the feedback reduces the error and the system reaches equilibrium.

The emitter resistor R12 limits the maximum motor current. The diode across the motor prevents any current from flowing from the positive end of the motor to the op amp, and to protect the transistor/circuit from damaging voltage spikes from the motor when power is cut.

I built this part of the circuit as such

In [10]: `InlineImage("motorConnectionsDiagram.jpg", 11,11, caption = "Figure 33: Motor Conne`

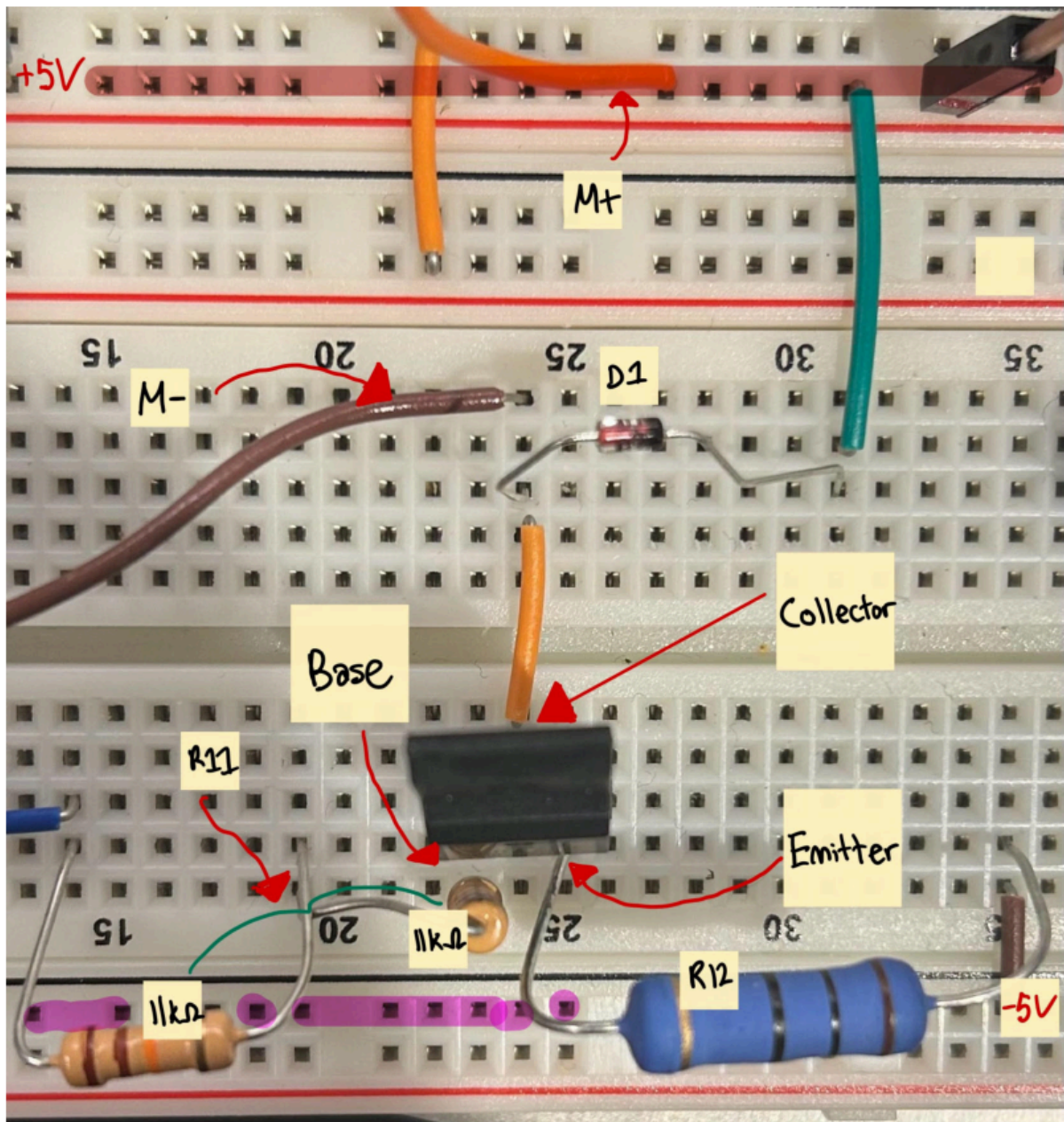


Figure 33: Motor Connection with BJT

Note: I used two $11k\Omega$ in series instead of a $22k\Omega$ resistor

TS

After connecting this final component, I attempted to test my entire circuit. I turned on the 5 hz wave gen into the inverter, and turned on my power supplies. I adjusted the potenitometer, but I found that the motor speed was not changing. I then realized that I was not getting the correct 5V output from my supplies, instead they were getting around 3V, which meant my AD3 was not able to supply enough power. I had my extra power brick connected into the AD3, so I knew that this had to be some type of wiring issue.

When I had tested the circuit with the error amplfiier, I did not see this issue, so I had the idea to disconnect the part of the circuit I had just built, with the BJT. When I did this, I saw that my voltage supplies were now able to supply their expected voltage. Now, I knew that the problem had to be with one of the componenets I had just added. So, I carefully checked that I had all the correct resistors, and that the BJT was connected correctly, with all the terminals going to the correct places. Here, I saw I had connected the diode the wrong way, which was likely why the circuit was demanding more current than my AD3 could supply.

After fixing it, my circuit worked as expected, with the motor speed changing with the potentiometer.

To properly test the circuit, I probed the + and - inputs of the error amplfifer, which gives V_{set} (the voltage set by the potenitometer), and $V_{in'}$ which is the latched analog value coming from the buffer, after it goes through a $1M\Omega$ resistor (both of these voltages can be seen in figure 29). In a properly fucntioning circuit, these voltages should follow each other, so as you change the voltage of the potentiometer, which represents changing the set speed of the mtoor, the analog converted latched value should follow, corresponding to the motor speeding up or slowing down to match the new set speed, which happens due to the error amplfifier generating a current according to how far off the set speed is from the current speed.

I noticed that they follow each other up until a certain point, after which they end up diverging. This makes sense, as the motor has a certain max speed, after which it cannot follow the set speed any longer, and it simply stays at its max speed.

In order to make a conversion table, showing the relationship between V_{set} at the potentiometer, and what RPM that voltage produces, I connected 8 DIO pins to the latch outputs, in order to see the latched value being stored, and I conected my scope to measure V_{set} .

I changed V_{set} in increments, and observed what my latched value was. I was able to produce this table.

V_set	Latch Value	RPM
0.25	20	600
0.30	30	900
0.40	42	1260
0.50	53	1590
0.55	58	1740
0.60	65	1950
0.70	75	2250
0.75	80	2400
0.80	85	2550
0.85	91	2730
0.90	96	2880
1.00	115	3450
1.14	122	3660
1.17	125	3750
1.20	128	3840
1.26	135	4050
1.30	140	4200
1.36	144	4320

Figure 34: Voltage to RPM

To convert the latched value to RPM, note that the latch input is driven by a 5 Hz pulse generator. This means the D-latch captures the counter value once every 0.2 s. The counter counts the number of holes passing the phototransistor during this 0.2 s time window. Since the disc has 10 holes, 10 pulses correspond to one full revolution of the motor shaft.

Therefore, the number of rotations occurring in 0.2 s is:

$$\frac{\text{Latch Value}}{10} = \text{rotations in 0.2s}$$

With some algebra

$$\text{RPS} = \frac{\text{Latch Value}}{10} * \frac{1}{0.2} = \frac{\text{Latch Value}}{2}$$

Multiplying by 60 to convert from seconds to minutes we get

$$\text{RPM} = \text{Latch Value} * 30$$

I found that the max speed of my motor was 4320 RPM, after which increasing V_{set} did not increase the RPM any further. So for $V_{set} \geq 1.36$, I had the same latched and RPM value as I do at $V_{set} = 1.36V$

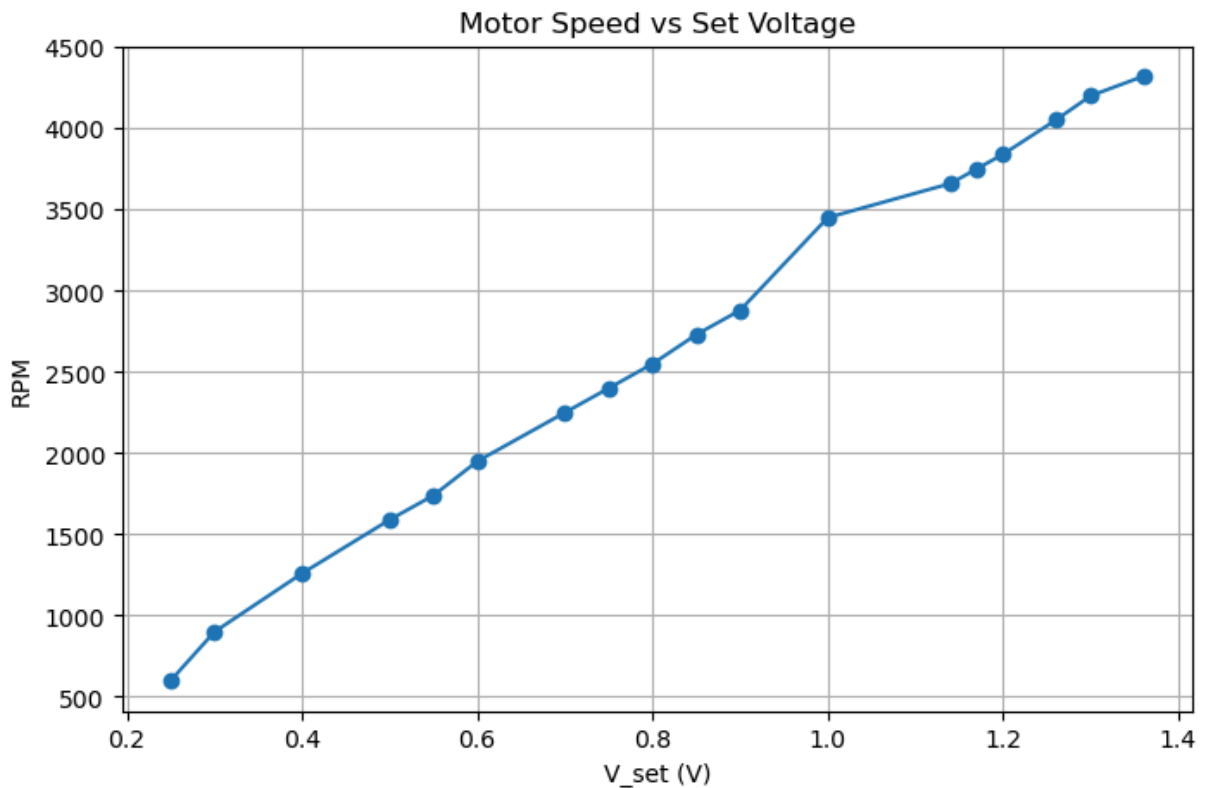
```
In [2]: import matplotlib.pyplot as plt

# Data
V_set = [0.25, 0.30, 0.40, 0.50, 0.55, 0.60, 0.70, 0.75, 0.80, 0.85,
         0.90, 1.00, 1.14, 1.17, 1.20, 1.26, 1.30, 1.36]

latch = [20, 30, 42, 53, 58, 65, 75, 80, 85, 91,
         96, 115, 122, 125, 128, 135, 140, 144]

# Compute RPM
RPM = [x * 30 for x in latch]

# Plot
plt.figure(figsize=(8,5))
plt.plot(V_set, RPM, marker='o')
plt.xlabel('V_set (V)')
plt.ylabel('RPM')
plt.title('Motor Speed vs Set Voltage')
plt.grid(True)
plt.show()
```



The relationship between V_{set} and RPM is linear (with some room for error, mainly one outlying data point), which once again confirms the motor is working correctly.

If we wanted to modify the circuit, such that the latched value shows the RPM, we can remember the formula derived early, which was

$$\text{Latch Value} = \frac{f_C}{f_I}$$

Where f_C is the frequency of the clock input (coming from the motor speed), and f_I is the frequency of the square wave at the inverter. We want the latched value to be 30 times larger to match the RPM, so this means we divide the frequency of the square input wave by 30, to be $\frac{1}{6}$ Hz. We would also have to add another counter, since currently we can only latch values of up to 255. If we added another 16 counter to this, we could store values of up to 4096.

Here is a picture of the entire circuit

```
In [33]: InlineImage("fullCircuit.jpg", 17,17, caption = "Figure 35: Full Circuit Photo ")
```

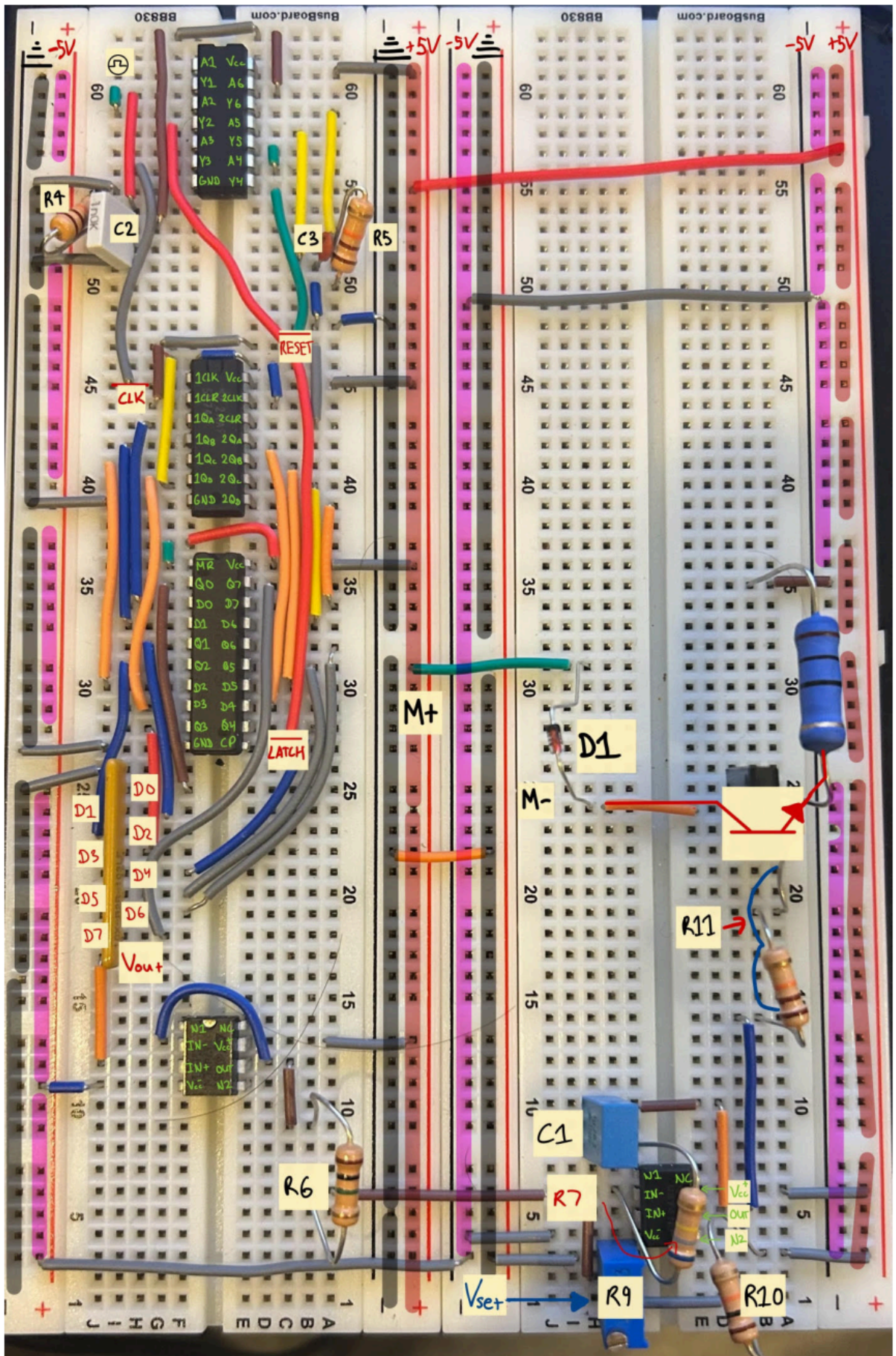


Figure 35: Full Circuit Photo

Conclusion

This project showed how digital logic and analog feedback can be combined to build a complete control system. Instead of treating the circuit as isolated blocks, the final design made it clear how different ideas from the course interconnect, logic gates create timing, counters convert pulses into a digital speed value, a DAC converts that value back into a voltage, and an op-amp integrates the error to set the motor current. Each individual circuit performs a simple function on its own, but together they form a controller capable of controlling motor speed.

A big takeaway is the importance of feedback in modern day systems. This controller circuit does not need an accurate model of the motor or the phototransistor, the relationship between V_{set} and RPM becomes linear simply by measuring the output and driving the input until the error reaches zero. We could likely use this same circuit (with some small adjustments) to control some component other than a motor, because the same feedback principles will apply. Control loops for speed, temperature, position, voltage all rely on the same idea. The project showed that complex control behavior can be built from basic components like latches, counters, op-amps, and a single transistor.

We often abstract these behaviors away. When designing a circuit like this, you do not have to understand the detailed physics behind a transistor, or how exactly a D-LATCH is built from different NAND gates. You simply have to understand the properties of the each component, and how they interact to create the system.